

1 Algorithm

1.1 LIS

```
1 // Longest increasing subsequence
2 int LIS(const vector<int>& s) {
3     // use binary search to update the smallest
4     // element
5     // that ends in a subsequence of length d
6     vector<int> v{};
7     v.push_back(s[0]);
8     for (int i{1}; i < s.size(); ++i)
9         if (s[i] > v.back()) v.push_back(s[i]);
10        else *lower_bound(v.begin(), v.end(),
11                          s[i]) = s[i];
12    return v.size();
13 }
```

1.2 LCS

```
1 // Longest common subsequence
2 int LCS(const vector<int>& a, const vector<
3         int>& b) {
4     vector<vector<int>>> dp(a.size() + 1,
5                             vector<int>(b.size() + 1));
6     for (int i{1}; i <= a.size(); ++i)
7         for (int j{1}; j <= b.size(); ++j) {
8             dp[i][j] = max(dp[i][j], max(dp[i - 1][j], dp[i][j - 1]));
9             if (a[i - 1] == b[j - 1]) dp[i][j] = max(dp[i][j], dp[i - 1][j - 1] + 1);
10        }
11    return dp[a.size()][b.size()];
12 }
```

2 Basic

2.1 mod helper function

```
1 int add(int i, int j) {
2     if ((i += j) >= MOD)
3         i -= MOD;
4     return i;
5 }
6
7 int sub(int i, int j) {
8     if ((i -= j) < 0)
9         i += MOD;
10    return i;
11 }
```

2.2 self-defined-pq-operator

```
1 auto cmp = [](int a, int b) {
2     return a > b;
3 };
4 priority_queue<int, vector<int>, decltype(
5     cmp)> pq(cmp);
```

2.3 generating all subsets

```
1 for (int b = 0; b < (1<<n); b++) {
2     vector<int> subset;
3     for (int i = 0; i < n; i++) {
4         if (b & (1<<i)) subset.push_back(v[i]);
5     }
6 }
```

2.4 memset

```
1 memset(a, 0, sizeof(a)); // 0
2 memset(a, 0x3f3f3f3f, sizeof(a)); // INF
```

2.5 submask enumeration

```
1 // O(3^n)
2
3 int m = 0b10110; // binary: 10110, decimal: 22
4
5 cout << "Mask: " << bitset<5>(m) << " (" <<
6     m << ")\\n";
7
8 // Enumerate all submasks
9 for (int s = m; s; s = (s - 1) & m) {
10    cout << bitset<5>(s) << " (" << s << ")\\n";
11 }
12
13 // Optionally include the empty submask
14 cout << bitset<5>(0) << " (0)\\n";
```

2.6 custom-hash

```
1 struct custom_hash {
2     static uint64_t splitmix64(uint64_t x) {
3         // <http://xorshift.di.unimi.it/splitmix64.c>
4         x += 0x9e3779b97f4a7c15;
5         x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
6         x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
```

```
7         return x ^ (x >> 31);
8     }
9
10    size_t operator()(uint64_t x) const {
11        static const uint64_t FIXED_RANDOM =
12            chrono::steady_clock::now().
13            time_since_epoch().count();
14        return splitmix64(x + FIXED_RANDOM);
15    }
16 };
17 unordered_map<long long, int, custom_hash>
18     safe_map;
19 gp_hash_table<long long, int, custom_hash>
20     safe_hash_table;
```

2.7 stringstream split by comma

```
1 while (std::getline(ss, segment, ',')) {
2     segments.push_back(segment);
3 }
```

3 DP

3.1 deque

```
1 /*
2 遊戲 DP - O(N^2)
3 A 與 B 將進行以下的遊戲。
4 最初，他們會得到一個序列 a = (a1, a2, ...,
5     aN)。在 a 尚未為空時，兩位玩家輪流進行以下
6     操作，從 A 開始：
7     從 a 的開頭或結尾移除一個元素。玩家會獲得 x
8     分，其中 x 為被移除的元素。
9     設 X 與 Y 分別為遊戲結束時 A 與 B 的總得分。
10    A 會嘗試最大化 X-Y，而 B 會嘗試最小化 X-Y。
11
12    假設兩位玩家都採取最優策略，請求出最後的 X-Y
13    值。
14
15    定義 dp[i][j] 為在區間 [i, j] 上，對於 B 來
16    說的最優分數 (X-Y)。
17 */
18 void solve() {
19     int n;
20     cin >> n;
21     vector<int> a(n);
22     vector<vector<int>>> dp(n + 1, vector<int>
23         (n + 1, 0));
24
25     for (int i = 0; i < n; ++i) {
26         cin >> a[i];
```

```
23         dp[i][i] = a[i];
24     }
25
26     for (int i = n - 1; i >= 0; --i) {
27         for (int j = i + 1; j < n; ++j) {
28             dp[i][j] = max(a[i] - dp[i + 1][j],
29                             a[j] - dp[i][j - 1]);
30         }
31     }
32     cout << dp[0][n - 1] << "\\n";
33 }
```

3.2 walk

```
1 /*
2 DP on graphs - O(N^3 Log K)
3 給定一個簡單的有向圖 G，具有 N 個頂點，編號
4     為 1, 2, ..., N。
5 對於任意 i, j (1 ≤ i, j ≤ N)，給定整數 a_{i,j}，
6     表示是否存在從頂點 i 指向頂點 j 的有向邊。若
7     a_{i,j} = 1，則存在邊；若 a_{i,j} = 0，則不存
8     在。
9 求圖中長度為 K 的不同有向路徑數目，對 10^9+7
10    取模。路徑可重複通過相同邊（即允許重複邊）。
11
12    注意：當我們將鄰接矩陣 m 與 m 相乘時，得到的
13    是長度為 2 的路徑數；若取 m 的 p 次方 m^p，
14    則其 (i, j) 元素表示從 i 到 j 的長度為 p 的
15    路徑數。
16 */
17 void solve() {
18     int n, k;
19     cin >> n >> k;
20     vector<vector<int>>> m(n, vector<int>(n));
21
22     for (int i = 0; i < n; ++i) {
23         for (int j = 0; j < n; ++j) {
24             cin >> m[i][j];
25         }
26     }
27
28     Matrix<int> mat(m);
29     mat = power(mat, k);
30     int ans = 0;
31     for (int i = 0; i < n; ++i) {
32         for (int j = 0; j < n; ++j) {
33             ans += mat[i][j];
34             ans %= MOD;
35         }
36     }
37     cout << ans << "\\n";
38 }
```

3.3 grouping

```

1  /*
2  狀態  $DP - O(3^N * 2^N * N^2)$ 
3  有  $N$  隻兔子，編號為  $1, 2, \dots, N$ 。
4
5  對於每一對  $i, j$  ( $1 \leq i, j \leq N$ )，兔子  $i$  與  $j$  的相容
   度由整數  $a_{i,j}$  描述。這裡  $a_{i,i} = 0$  對於
   每個  $i$  ( $1 \leq i \leq N$ )，且  $a_{i,j} = a_{j,i}$  對於任
   意  $i$  與  $j$  ( $1 \leq i, j \leq N$ )。
6
7  A 將  $N$  隻兔子分成若干個群組。每隻兔子必須且
   僅屬於一個群組。分群後，對於每一對  $i$  與
    $j$  ( $1 \leq i < j \leq N$ )，若兔子  $i$  與  $j$  屬於同一群
   組，A 即可獲得  $a_{i,j}$  分。
8
9  求 A 能獲得的最大總分。
10
11 令  $cost[S]$  表示將集合  $S$  中的所有兔子放在同一
   群組時所得到的分數。此值可在  $O(2^N * N^2)$  時間內計算。
12
13 接著我們計算  $dp[S]$ ，表示對集合  $S$  中的兔子進
   行分群時所能得到的最大分數。
14
15 */
16 void solve() {
17     int n;
18     cin >> n;
19     vector<vector<int>>> a(n, vector<int>(n));
20     ;
21     vector<int> cost(1<<n, 0);
22     vector<int> dp(1<<n, 0);
23
24     for (int i = 0; i < n; ++i) {
25         for (int j = 0; j < n; ++j) {
26             cin >> a[i][j];
27         }
28     }
29
30     // backtrack all subset
31     for (int b = 0; b < (1<<n); ++b) {
32         vector<int> subset;
33         for (int i = 0; i < n; ++i) {
34             if (b & (1<<i)) {
35                 for (const int& j : subset) {
36                     cost[b] += a[i][j];
37                 }
38             }
39         }
40     }
41
42     // dp
43     for (int i = 0; i < (1<<n); ++i) {
44         int j = ((1<<n) - 1) ^ i;
45         for (int s = j; s != 0; s = (s - 1)
46             & j) {
47             dp[i ^ s] = max(dp[i ^ s], dp[i]
48                 + cost[s]);
49         }
50     }

```

3.4 matching

```

48     }
49     cout << dp[(1<<n) - 1] << "\n";
50 }

```

```

1  /*
2  Bitmask DP -  $O(N * 2^N)$ 
3  有  $N$  個男人和  $N$  個女人，分別編號為  $1, 2, \dots, N$ 。
4
5  對於每個  $i, j$  ( $1 \leq i, j \leq N$ )，男人  $i$  和女人
    $j$  的相容性由整數  $a[i][j]$  給出。
6  如果  $a[i][j] = 1$ ，則男人  $i$  和女人  $j$  是相容
   的；
7  如果  $a[i][j] = 0$ ，則不是。
8
9  A 正在嘗試組成  $N$  對，每對由一個相容的男人和
   女人組成。在這裡，每個男人和每個女人必須
   恰好屬於一對。
10
11 求 A 可以組成  $N$  對的方法數，結果對  $10^9 + 7$ 
   取模。
12
13 定義  $dp[S]$  為將集合  $S$  中的女性與前  $|S|$  個男
   性配對的方法數。
14
15 */
16 const int maxn = 21;
17 const int MOD = 1e9 + 7;
18 int n;
19 int grid[maxn][maxn];
20 int dp[1<<maxn];
21
22 void solve() {
23     cin >> n;
24     memset(dp, 0, sizeof(dp));
25
26     for (int i = 0; i < n; ++i) {
27         for (int j = 0; j < n; ++j) {
28             cin >> grid[i][j];
29         }
30     }
31
32     dp[0] = 1;
33     for (int s = 0; s < (1<<n); ++s) {
34         int ps = __builtin_popcount(s);
35         for (int w = 0; w < n; ++w) {
36             if ((s & (1<<w)) || !grid[ps][w]) {
37                 continue;
38             }
39
40             dp[s | (1<<w)] += dp[s];
41             dp[s | (1<<w)] %= MOD;
42         }
43     }
44     cout << dp[(1<<n) - 1] << "\n";
45 }

```

3.5 projects

```

1  /*
2  LIS DP -  $O(N \log N)$ 
3  有  $n$  個你可以參加的專案。對於每個專案，你知
   道其開始與結束天數以及可獲得的報酬金額。
4  在同一天你最多只能參加一個專案。
5  問：你最多可以賺到多少金額？
6
7   $dp[i]$  = 在第  $i$  天之前我們可以賺到的最大金
   額。
8
9  */
10 void solve() {
11     int n;
12     cin >> n;
13     vector<array<int, 3>> vc(n);
14     map<int, int> days;
15
16     for (int i = 0; i < n; ++i) {
17         int a, b, p;
18         cin >> a >> b >> p;
19         days[a] = days[b] = 1;
20         vc[i] = {a, b, p};
21     }
22     int idx = 1;
23     for (auto& x : days) {
24         x.second = idx++;
25     }
26     vector<int> dp(idx, 0);
27
28     sort(vc.begin(), vc.end(), [](const
29         array<int, 3>& va, const array<int,
30         3>& vb) {
31         if (va[1] != vb[1]) return va[1] <
32             vb[1];
33         if (va[0] != vb[0]) return va[0] <
34             vb[0];
35         return va[2] > vb[2];
36     });
37
38     int i = 0;
39     for (int d = 1; d < idx; ++d) {
40         dp[d] = dp[d - 1];
41         while (i < n && days[vc[i][1]] == d) {
42             dp[d] = max(dp[d], dp[days[vc[i]
43                 ][0]] - 1 + vc[i][2]);
44             i++;
45         }
46     }
47     cout << dp[idx - 1] << "\n";
48 }

```

3.6 stones

```

1  /*
2  遊戲  $DP - O(N^2)$ 
3  有一個集合  $A = \{a_1, a_2, \dots, a_N\}$ ，包含  $N$  個正整
   數。太郎和次郎將進行以下的遊戲。
4

```

```

5 一開始有一堆  $K$  顆石頭。兩位玩家輪流進行以下
   操作，從太郎開始：
6
7 選擇集合  $A$  中的一個元素  $x$ ，並從石堆中移除恰
   好  $x$  顆石頭。
8 當某位玩家無法進行操作時即輸掉比賽。假設兩位
   玩家都採取最優策略，請判斷誰會獲勝。
9
10 定義  $dp[i]$  表示當剩下  $i$  顆石頭時，是否有可能
   獲勝。
11
12 */
13 void solve() {
14     int n, k;
15     cin >> n >> k;
16     vector<int> a(n);
17     vector<bool> dp(k + 1, 0);
18
19     for (int i = 0; i < n; ++i) {
20         cin >> a[i];
21     }
22
23     for (int i = 1; i <= k; ++i) {
24         for (int x : a) {
25             if (i >= x && !dp[i - x]) {
26                 dp[i] = 1;
27             }
28         }
29     }
30     cout << (dp[k] ? "First" : "Second") <<
31         "\n";
32 }

```

3.7 coins

```

1  /*
2  機率  $DP - O(N^2)$ 
3  給定一個正奇數  $N$ 
4  有  $N$  枚編號為  $1, 2, \dots, N$  的硬幣，第  $i$  枚出現正
   面的機率為  $p$ ，反面為  $1 - p$ 。
5  已經拋擲所有硬幣，求正面數大於反面的機率。
6
7  定義  $dp[i][j]$  為拋完前  $i$  枚硬幣後，得到  $j$  次
   正面的機率。
8
9  */
10 void solve() {
11     int n;
12     cin >> n;
13     vector<double> a(n);
14     vector<vector<double>> dp(n + 1, vector<
15         double>(n + 1, 0.0));
16
17     for (int i = 0; i < n; ++i) {
18         cin >> a[i];
19     }
20
21     for (int i = 0; i <= n; ++i) {
22         dp[i][0] = 1.0;
23     }

```

```

24 int least = n / 2 + 1;
25
26 for (int i = 1; i <= n; ++i) {
27     for (int j = 1; j <= least; ++j) {
28         // head
29         dp[i][j] = dp[i - 1][j - 1] * a[
30             i - 1];
31         // tail
32         dp[i][j] += dp[i - 1][j] * (1 -
33             a[i - 1]);
34     }
35 }
36
37 cout << fixed << setprecision(10) << dp[
38     n][least] << "\n";

```

3.8 elevator rides

```

1 /*
2 狀態 DP -  $O(2^N)$ 
3 有  $n$  個人想要搭電梯到樓頂。建築物只有一部電
4 梯。你知道每個人的體重以及電梯的最大允許
5 載重。最少需要搭乘多少次電梯？
6
7 定義  $dp[S] = \{r, w\}$ 。其中  $r$  是將集合  $S$  中的
8 所有人送到樓頂所需的最少電梯次數。而  $w$  是最
9 後一次電梯所載人的總重量。
10
11 */
12
13 void solve() {
14     int n, x;
15     cin >> n >> x;
16     vector<int> w(n);
17     vector<pii> dp(1<<n, {INF, INF});
18
19     for (int i = 0; i < n; ++i) {
20         cin >> w[i];
21     }
22
23     dp[0] = {1, 0};
24     for (int b = 1; b < (1<<n); ++b) {
25         for (int i = 0; i < n; ++i) {
26             if (b & (1<<i)) {
27                 auto [r_prev, w_prev] = dp[b
28                     ^ (1<<i)];
29                 pii can;
30                 if (w_prev + w[i] <= x) {
31                     can = {r_prev, w_prev +
32                         w[i]};
33                 }
34                 else {
35                     can = {r_prev + 1, w[i]
36                         };
37                 }
38                 dp[b] = min(dp[b], can);
39             }
40         }
41     }
42     cout << dp[(1<<n) - 1].first << "\n";
43 }

```

3.9 slimes

```

1 /*
2 Range DP -  $O(N^3)$ 
3 有  $N$  個史萊姆排成一列。最初，從左邊數來第  $i$ 
4 個史萊姆的大小為  $a_i$ 。
5
6 A 想要把所有史萊姆合併成一個更大的史萊姆。他
7 會重複執行以下操作，直到只剩下一個史萊姆
8 為止：
9 選擇兩個相鄰的史萊姆，將它們合併成一個新的史
10 萊姆。新史萊姆的大小為  $x+y$ ，其中  $x$  和  $y$ 
11 是合併前兩個史萊姆的大小。
12 這時會產生  $x+y$  的花費。合併時，史萊姆的相對
13 位置不會改變。
14 請求出合併所有史萊姆所需的最小總花費。
15
16 令  $dp[i][j]$  表示將第  $i$  個到第  $j$  個史萊姆合併
17 成一個史萊姆的最小花費。
18
19 */
20
21 const int maxn = 401;
22 const int INF = 1e18;
23 int dp[maxn][maxn];
24 int a[maxn];
25 int prefix[maxn + 1];
26
27 int f(int i, int j) {
28     if (i + 1 == j) {
29         return a[i] + a[j];
30     }
31     if (i == j) {
32         return 0;
33     }
34     if (dp[i][j] != INF) {
35         return dp[i][j];
36     }
37     //cerr << i << " " << j << "\n";
38
39     int ans = INF;
40     for (int k = i; k < j; ++k) {
41         ans = min(ans, f(i, k) + f(k + 1, j)
42             );
43     }
44     return dp[i][j] = ans + (prefix[j + 1] -
45         prefix[i]);
46 }

```

3.10 digit sum

```

1 /*
2 Digit DP -  $O(|K| * D)$ 
3 計算在  $1$  到  $K$  (含) 之間，滿足其十進位數字和
4 為  $D$  的倍數的整數數量。答案對  $10^9+7$  取
5 模。
6
7 令  $dp[i][j]$  表示在已確定前  $i$  位數字的情況
8 下，構成長度為  $|K|$  的數字且目前數字和

```

```

1 mod  $D$  等於  $j$  的方法數。
2
3 */
4
5 const int MOD = 1e9 + 7;
6 int dp[10001][101][2];
7
8 void solve() {
9     string K;
10     int D;
11     cin >> K >> D;
12     int len = K.size();
13     memset(dp, 0, sizeof(dp));
14     dp[0][0][1] = 1;
15
16     for (int i = 1; i <= len; ++i) {
17         int limit = K[i - 1] - '0';
18         for (int s = 0; s < D; ++s) {
19             for (int flag = 0; flag <= 1; ++
20                 flag) {
21                 int ways = dp[i - 1][s][flag
22                     ];
23                 if (ways == 0) continue;
24                 int max_d = (flag ? limit :
25                     9);
26                 for (int d = 0; d <= max_d;
27                     ++d) {
28                     int rs = (s + d) % D;
29                     int rflag = (flag && d
30                         == max_d ? 1 : 0);
31                     dp[i][rs][rflag] += ways
32                         ;
33                     dp[i][rs][rflag] %= MOD;
34                 }
35             }
36         }
37     }
38
39     int ans = (dp[len][0][0] + dp[len
40         ][0][1]) % MOD;
41     ans = (ans - 1 + MOD) % MOD;
42     cout << ans << "\n";
43 }

```

3.11 sushi

```

1 /*
2 期望值 DP -  $O(N^3)$ 
3 有  $N$  盤壽司，編號從  $1$  到  $N$ 。第  $i$  盤最初有  $a_i$ 
4 ( $1 \leq a_i \leq 3$ ) 塊壽司。
5
6 太郎不斷擲一顆編號  $1$  到  $N$  的骰子。如果結果是
7 第  $i$  盤且該盤還有壽司，他就吃掉一塊；否
8 則什麼也不做。
9
10 請求出吃完所有壽司所需擲骰子的期望次數。
11
12  $dp[x][y][z]$  代表還有
13  $x$  盤剩 1 塊壽司、
14  $y$  盤剩 2 塊壽司、
15  $z$  盤剩 3 塊壽司時的期望擲骰次數。
16
17 */

```

```

14 const int maxn = 301;
15 double dp[maxn][maxn][maxn];
16 int n;
17
18 double dfs(int x, int y, int z) {
19     if (x < 0 || y < 0 || z < 0) return 0;
20     if (x == 0 && y == 0 && z == 0) return
21         0;
22     if (dp[x][y][z] > 0) return dp[x][y][z];
23     double ans = n + x * dfs(x - 1, y, z)
24         + y * dfs(x + 1, y - 1, z
25             )
26         + z * dfs(x, y + 1, z -
27             1);
28     return dp[x][y][z] = ans / (x + y + z);
29 }
30
31 void solve() {
32     cin >> n;
33     vector<int> a(n);
34     memset(dp, -1, sizeof(dp));
35     vector<int> freq(4, 0);
36
37     for (int i = 0; i < n; ++i) {
38         cin >> a[i];
39         freq[a[i]]++;
40     }
41
42     cout << fixed << setprecision(10) << dfs
43         (freq[1], freq[2], freq[3]) << "\n";
44 }

```

3.12 candies

```

1 /*
2 組合 DP -  $O(NK)$ 
3 有  $N$  個小孩，編號為  $1, 2, \dots, N$ 。
4
5 他們決定將  $K$  顆糖果分給自己。對於每個  $i$  ( $1 \leq i \leq N$ )，第  $i$  個小孩最多可以拿到  $a_i$  顆糖果
6 (包含  $0$  顆)。所有糖果都必須分完，不能
7 剩下。
8
9 請問有多少種分配糖果的方法？請將答案對
10  $10^9+7$  取模。若存在某個小孩分到的糖果數
11 不同，則視為不同的分配方式。
12
13 令  $dp[i][j]$  表示將  $j$  顆糖果分給前  $i$  個小孩的
14 方法數。
15
16 */
17
18 void solve() {
19     int n, k;
20     cin >> n >> k;
21     vector<int> a(n);
22     vector<int> dp(k + 1, 0), S(k + 1, 0);
23
24     for (int i = 0; i < n; ++i) {
25         cin >> a[i];
26     }

```

```

21 dp[0] = 1;
22 for (int i = 0; i < n; ++i) {
23     vector<int> new_dp(k + 1, 0);
24     S[0] = dp[0];
25     for (int j = 1; j <= k; ++j) {
26         S[j] = (S[j - 1] + dp[j]) % MOD;
27     }
28     for (int j = 0; j <= k; ++j) {
29         if (j - a[i] - 1 >= 0) {
30             new_dp[j] = (S[j] - S[j - a[i] - 1] + MOD) % MOD;
31         }
32         else {
33             new_dp[j] = S[j] % MOD;
34         }
35     }
36     dp = new_dp;
37 }
38 cout << dp[k] << "\n";
39 }
40 }

```

3.13 permutation

```

1  /*
2  抽象 DP -  $O(N^2)$ 
3  設  $N$  為正整數。給定一個長度為  $N-1$  的字串  $s$ ，
4  字元僅包含 ' $<$ ' 與 ' $>$ '。
5  求滿足條件的排列  $(p_1, p_2, \dots, p_N)$  (即 1 到  $N$ 
6  的排列) 數量。答案對  $10^9+7$  取模：
7  對於每個  $i (1 \leq i \leq N-1)$ ，若  $s$  的第  $i$  個字
8  元為 ' $<$ '，則要求  $p_i < p_{i+1}$ ；若為 ' $>$ '，
9  則要求  $p_i > p_{i+1}$ 。
10 */
11 void solve() {
12     int n;
13     string s;
14     cin >> n >> s;
15     vector<vector<int>> dp(n + 1, vector<int>
16         >(n + 1, 0));
17     vector<int> prefix(n + 1, 0);
18     dp[1][0] = 1;
19     for (int i = 2; i <= n; ++i) {
20         for (int k = 0; k < n; ++k) {
21             prefix[k + 1] = prefix[k] + dp[i
22                 - 1][k];
23         }
24         for (int j = 0; j < i; ++j) {
25             if (s[i - 2] == '>') {
26                 dp[i][j] += prefix[i - 1] -
27                     prefix[j];
28             }
29             else {
30                 dp[i][j] += prefix[j];
31             }
32         }
33     }
34     cout << dp[n][0] << "\n";
35 }

```

```

29     for (int k = j; k < i - 1;
30         ++k) {
31         dp[i][j] += dp[i - 1][k];
32     }
33     }
34     else {
35         dp[i][j] += prefix[j];
36         dp[i][j] %= MOD;
37     }
38     for (int k = 0; k < j; ++k) {
39         dp[i][j] += dp[i - 1][k];
40     }
41     }
42     }
43     }
44     }
45     int ans = 0;
46     for (int j = 0; j < n; ++j) {
47         ans += dp[n][j];
48         ans %= MOD;
49     }
50     cout << ans << "\n";
51 }

```

3.14 Knasack2

```

1  // 01 背包，背包承重大 ( $1e9$ )，物品價值和較小
2  // ( $1e5$ )
3  const int maxn = 101;
4  const int maxv = 100001;
5  int weight[maxn];
6  int cost[maxn];
7  int dp[maxv];
8  void solve() {
9     int n, w;
10     cin >> n >> w;
11     for (int i = 0; i < n; ++i) {
12         cin >> weight[i] >> cost[i];
13     }
14     fill(dp, dp + maxv, 1e18);
15     dp[0] = 0;
16     for (int i = 0; i < n; ++i) {
17         for (int j = maxv - 1; j >= 0; --j) {
18             if (dp[j] + weight[i] <= w) {
19                 dp[j + cost[i]] = min(dp[j +
20                     cost[i]], dp[j] +
21                     weight[i]);
22             }
23         }
24     }
25     for (int i = maxv - 1; i >= 0; --i) {
26         if (dp[i] != 1e18) {
27             cout << i << "\n";
28         }
29     }

```

```

30     return;
31     }
32     }
33 }
34 }
35 }
36 }
37 }
38 }
39 }
40 }
41 }
42 }
43 }
44 }
45 }
46 }
47 }
48 }
49 }
50 }
51 }
52 }
53 }
54 }
55 }
56 }
57 }
58 }
59 }
60 }
61 }
62 }
63 }
64 }
65 }
66 }
67 }
68 }
69 }
70 }
71 }
72 }
73 }
74 }
75 }
76 }
77 }
78 }
79 }
80 }
81 }
82 }
83 }
84 }
85 }
86 }
87 }
88 }
89 }
90 }
91 }
92 }
93 }
94 }
95 }
96 }
97 }
98 }
99 }
100 }
101 }
102 }
103 }
104 }
105 }
106 }
107 }
108 }
109 }
110 }
111 }
112 }
113 }
114 }
115 }
116 }
117 }
118 }
119 }
120 }
121 }
122 }
123 }
124 }
125 }
126 }
127 }
128 }
129 }
130 }
131 }
132 }
133 }
134 }
135 }
136 }
137 }
138 }
139 }
140 }
141 }
142 }
143 }
144 }
145 }
146 }
147 }
148 }
149 }
150 }
151 }
152 }
153 }
154 }
155 }
156 }
157 }
158 }
159 }
160 }
161 }
162 }
163 }
164 }
165 }
166 }
167 }
168 }
169 }
170 }
171 }
172 }
173 }
174 }
175 }
176 }
177 }
178 }
179 }
180 }
181 }
182 }
183 }
184 }
185 }
186 }
187 }
188 }
189 }
190 }
191 }
192 }
193 }
194 }
195 }
196 }
197 }
198 }
199 }
200 }
201 }
202 }
203 }
204 }
205 }
206 }
207 }
208 }
209 }
210 }
211 }
212 }
213 }
214 }
215 }
216 }
217 }
218 }
219 }
220 }
221 }
222 }
223 }
224 }
225 }
226 }
227 }
228 }
229 }
230 }
231 }
232 }
233 }
234 }
235 }
236 }
237 }
238 }
239 }
240 }
241 }
242 }
243 }
244 }
245 }
246 }
247 }
248 }
249 }
250 }
251 }
252 }
253 }
254 }
255 }
256 }
257 }
258 }
259 }
260 }
261 }
262 }
263 }
264 }
265 }
266 }
267 }
268 }
269 }
270 }
271 }
272 }
273 }
274 }
275 }
276 }
277 }
278 }
279 }
280 }
281 }
282 }
283 }
284 }
285 }
286 }
287 }
288 }
289 }
290 }
291 }
292 }
293 }
294 }
295 }
296 }
297 }
298 }
299 }
300 }
301 }
302 }
303 }
304 }
305 }
306 }
307 }
308 }
309 }
310 }
311 }
312 }
313 }
314 }
315 }
316 }
317 }
318 }
319 }
320 }
321 }
322 }
323 }
324 }
325 }
326 }
327 }
328 }
329 }
330 }
331 }
332 }
333 }
334 }
335 }
336 }
337 }
338 }
339 }
340 }
341 }
342 }
343 }
344 }
345 }
346 }
347 }
348 }
349 }
350 }
351 }
352 }
353 }
354 }
355 }
356 }
357 }
358 }
359 }
360 }
361 }
362 }
363 }
364 }
365 }
366 }
367 }
368 }
369 }
370 }
371 }
372 }
373 }
374 }
375 }
376 }
377 }
378 }
379 }
380 }
381 }
382 }
383 }
384 }
385 }
386 }
387 }
388 }
389 }
390 }
391 }
392 }
393 }
394 }
395 }
396 }
397 }
398 }
399 }
400 }
401 }
402 }
403 }
404 }
405 }
406 }
407 }
408 }
409 }
410 }
411 }
412 }
413 }
414 }
415 }
416 }
417 }
418 }
419 }
420 }
421 }
422 }
423 }
424 }
425 }
426 }
427 }
428 }
429 }
430 }
431 }
432 }
433 }
434 }
435 }
436 }
437 }
438 }
439 }
440 }
441 }
442 }
443 }
444 }
445 }
446 }
447 }
448 }
449 }
450 }
451 }
452 }
453 }
454 }
455 }
456 }
457 }
458 }
459 }
460 }
461 }
462 }
463 }
464 }
465 }
466 }
467 }
468 }
469 }
470 }
471 }
472 }
473 }
474 }
475 }
476 }
477 }
478 }
479 }
480 }
481 }
482 }
483 }
484 }
485 }
486 }
487 }
488 }
489 }
490 }
491 }
492 }
493 }
494 }
495 }
496 }
497 }
498 }
499 }
500 }
501 }
502 }
503 }
504 }
505 }
506 }
507 }
508 }
509 }
510 }
511 }
512 }
513 }
514 }
515 }
516 }
517 }
518 }
519 }
520 }
521 }
522 }
523 }
524 }
525 }
526 }
527 }
528 }
529 }
530 }
531 }
532 }
533 }
534 }
535 }
536 }
537 }
538 }
539 }
540 }
541 }
542 }
543 }
544 }
545 }
546 }
547 }
548 }
549 }
550 }
551 }
552 }
553 }
554 }
555 }
556 }
557 }
558 }
559 }
560 }
561 }
562 }
563 }
564 }
565 }
566 }
567 }
568 }
569 }
570 }
571 }
572 }
573 }
574 }
575 }
576 }
577 }
578 }
579 }
580 }
581 }
582 }
583 }
584 }
585 }
586 }
587 }
588 }
589 }
590 }
591 }
592 }
593 }
594 }
595 }
596 }
597 }
598 }
599 }
600 }
601 }
602 }
603 }
604 }
605 }
606 }
607 }
608 }
609 }
610 }
611 }
612 }
613 }
614 }
615 }
616 }
617 }
618 }
619 }
620 }
621 }
622 }
623 }
624 }
625 }
626 }
627 }
628 }
629 }
630 }
631 }
632 }
633 }
634 }
635 }
636 }
637 }
638 }
639 }
640 }
641 }
642 }
643 }
644 }
645 }
646 }
647 }
648 }
649 }
650 }
651 }
652 }
653 }
654 }
655 }
656 }
657 }
658 }
659 }
660 }
661 }
662 }
663 }
664 }
665 }
666 }
667 }
668 }
669 }
670 }
671 }
672 }
673 }
674 }
675 }
676 }
677 }
678 }
679 }
680 }
681 }
682 }
683 }
684 }
685 }
686 }
687 }
688 }
689 }
690 }
691 }
692 }
693 }
694 }
695 }
696 }
697 }
698 }
699 }
700 }
701 }
702 }
703 }
704 }
705 }
706 }
707 }
708 }
709 }
710 }
711 }
712 }
713 }
714 }
715 }
716 }
717 }
718 }
719 }
720 }
721 }
722 }
723 }
724 }
725 }
726 }
727 }
728 }
729 }
730 }
731 }
732 }
733 }
734 }
735 }
736 }
737 }
738 }
739 }
740 }
741 }
742 }
743 }
744 }
745 }
746 }
747 }
748 }
749 }
750 }
751 }
752 }
753 }
754 }
755 }
756 }
757 }
758 }
759 }
760 }
761 }
762 }
763 }
764 }
765 }
766 }
767 }
768 }
769 }
770 }
771 }
772 }
773 }
774 }
775 }
776 }
777 }
778 }
779 }
780 }
781 }
782 }
783 }
784 }
785 }
786 }
787 }
788 }
789 }
790 }
791 }
792 }
793 }
794 }
795 }
796 }
797 }
798 }
799 }
800 }
801 }
802 }
803 }
804 }
805 }
806 }
807 }
808 }
809 }
810 }
811 }
812 }
813 }
814 }
815 }
816 }
817 }
818 }
819 }
820 }
821 }
822 }
823 }
824 }
825 }
826 }
827 }
828 }
829 }
830 }
831 }
832 }
833 }
834 }
835 }
836 }
837 }
838 }
839 }
840 }
841 }
842 }
843 }
844 }
845 }
846 }
847 }
848 }
849 }
850 }
851 }
852 }
853 }
854 }
855 }
856 }
857 }
858 }
859 }
860 }
861 }
862 }
863 }
864 }
865 }
866 }
867 }
868 }
869 }
870 }
871 }
872 }
873 }
874 }
875 }
876 }
877 }
878 }
879 }
880 }
881 }
882 }
883 }
884 }
885 }
886 }
887 }
888 }
889 }
890 }
891 }
892 }
893 }
894 }
895 }
896 }
897 }
898 }
899 }
900 }
901 }
902 }
903 }
904 }
905 }
906 }
907 }
908 }
909 }
910 }
911 }
912 }
913 }
914 }
915 }
916 }
917 }
918 }
919 }
920 }
921 }
922 }
923 }
924 }
925 }
926 }
927 }
928 }
929 }
930 }
931 }
932 }
933 }
934 }
935 }
936 }
937 }
938 }
939 }
940 }
941 }
942 }
943 }
944 }
945 }
946 }
947 }
948 }
949 }
950 }
951 }
952 }
953 }
954 }
955 }
956 }
957 }
958 }
959 }
960 }
961 }
962 }
963 }
964 }
965 }
966 }
967 }
968 }
969 }
970 }
971 }
972 }
973 }
974 }
975 }
976 }
977 }
978 }
979 }
980 }
981 }
982 }
983 }
984 }
985 }
986 }
987 }
988 }
989 }
990 }
991 }
992 }
993 }
994 }
995 }
996 }
997 }
998 }
999 }
1000 }

```

3.15 flowers

```

1  /*
2  LIS DP + Segment Tree -  $O(N \log N)$ 
3  有  $N$  朵花排成一列。對於每個  $i (1 \leq i \leq N)$ ，
4  第  $i$  朵花的高度與美麗分別為  $h_i$  與  $a_i$ 。
5  此處  $h_1, h_2, \dots, h_N$  兩兩互異。
6  A 會拔掉一些花，使得剩下的花從左到右的高度為
7  單調遞增 (嚴格遞增)。
8  求剩下花的美麗值總和的最大可能值。
9  令  $dp[i]$  表示以第  $i$  朵花為結尾的遞增子序列所
10  能取得的最大美麗值。
11  */
12 void solve() {
13     int n;
14     cin >> n;
15     SGT<int, MergeMax> tree(n + 1, 0);
16     vector<int> h(n), b(n);
17     for (int i = 0; i < n; ++i) {
18         cin >> h[i];
19     }
20     for (int i = 0; i < n; ++i) {
21         cin >> b[i];
22     }
23     for (int i = 0; i < n; ++i) {
24         int mx = tree.query(0, h[i]);
25         tree.modify(h[i], mx + b[i]);
26     }
27     cout << tree.query(0, n + 1) << "\n";
28 }

```

3.16 independent set

```

1  /*
2  DP on Trees -  $O(N)$ 
3  有一棵含  $N$  個頂點的樹，頂點編號為  $1, 2, \dots, N$ 。
4  對於每個  $i (1 \leq i \leq N-1)$ ，第  $i$  條邊連接
5  頂點  $x_i$  和  $y_i$ 。
6  A 決定將每個頂點塗成白色或黑色，但不允許兩個
7  相鄰的頂點同時為黑色。
8  求將頂點塗色的方案數，對  $10^9+7$  取模。
9  */

```

```

9  設  $dp[i][j]$  表示以節點  $i$  為根的子樹中，在節
10  點  $i$  顏色為  $j$  時的塗色方案數 (例如  $j=0$ 
11  表示白色， $j=1$  表示黑色)。
12  */
13 const int maxn = 100001;
14 vector<int> adj[maxn];
15 int f[maxn][2];
16 const int MOD = 1e9 + 7;
17 void dp(int u, int p) {
18     for (int v : adj[u]) {
19         if (v != p) {
20             dp(v, u);
21             f[u][0] = (f[v][0] + f[v][1]) %
22                 MOD * f[u][0] % MOD;
23             f[u][1] = f[v][0] * f[u][1] %
24                 MOD;
25         }
26     }
27     void solve() {
28         int n;
29         cin >> n;
30         for (int i = 0; i < n; ++i) {
31             f[i][0] = f[i][1] = 1;
32         }
33         for (int i = 0; i < n - 1; ++i) {
34             int u, v;
35             cin >> u >> v;
36             u--, v--;
37             adj[u].pb(v);
38             adj[v].pb(u);
39         }
40         dp(0, -1);
41         cout << (f[0][0] + f[0][1]) % MOD << "\n";
42     }
43 }

```

3.17 counting tower

```

1  /*
2  狀態機 DP -  $O(N)$ 
3  你的任務是建造一座寬度為 2、高度為  $n$  的塔。
4  你有無限數量寬度與高度為整數的方塊。
5   $dp[i][0]$  = 高度為  $i$  的塔中，頂層為一個寬度為
6  2 的方塊 (即該層由跨越兩欄的單一方塊覆
7  蓋) 的塔的数量。
8   $dp[i][1]$  = 高度為  $i$  的塔中，頂層在該層有兩個
9  寬度為 1 的方塊 (每欄各一個) 的塔的数量。
10 */
11 void solve() {

```

```

10 long long n;
11 cin >> n;
12 dp[1][0] = 1;
13 dp[1][1] = 1;
14 for(int i = 2; i <= n; i++){
15     dp[i][0] = (4 * dp[i-1][0] + dp[i-1][1]) % mod;
16     dp[i][1] = (dp[i-1][0] + 2 * dp[i-1][1]) % mod;
17 }
18 cout << (dp[n][0] + dp[n][1]) % mod << endl;
19 }

```

3.18 counting numbers

```

1 /*
2 區間 DP - O(digits*10)
3 您的任務是計算在區間 a 到 b 之間，沒有任何相鄰兩位數字相同的整數的個數。
4
5 定義 dp[pos][prev_digit][is_tight][is_started] 為從位置 pos 到結尾的有效整數數量。
6 其中 prev_digit 是位置 pos-1 的數字。
7 is_tight 表示是否受原數字前綴的限制。
8 is_started 表示是否已開始構成有效數字（用以避免計入前導零）。
9 */
10
11 // dp[pos][prev_digit][is_tight][is_started]
12 int dp[20][10][2][2];
13 string s;
14
15 int f(int pos, int prev_digit, bool is_tight, bool is_started) {
16     if (pos == (int)s.size()) {
17         return 1;
18     }
19     if (dp[pos][prev_digit][is_tight][is_started] != -1) {
20         return dp[pos][prev_digit][is_tight][is_started];
21     }
22
23     int ans = 0;
24     int max_d = is_tight ? (s[pos] - '0') : 9;
25     for (int d = 0; d <= max_d; ++d) {
26         if (is_started && d == prev_digit) {
27             continue;
28         }
29
30         bool new_is_started = is_started || (d > 0);
31         bool new_is_tight = is_tight && (d == max_d);
32         ans += f(pos + 1, d, new_is_tight, new_is_started);
33     }
34 }

```

```

35 return dp[pos][prev_digit][is_tight][is_started] = ans;
36 }
37
38 int count(int x) {
39     if (x < 0) return 0;
40     s = to_string(x);
41     memset(dp, -1, sizeof(dp));
42     return f(0, 0, true, false);
43 }
44
45 void solve() {
46     int a, b;
47     cin >> a >> b;
48
49     int ca = count(a - 1);
50     int cb = count(b);
51     cout << cb - ca << "\n";
52 }

```

4 Data Structure

4.1 undo disjoint set

```

1 struct DisjointSet {
2     // save() is like recursive
3     // undo() is like return
4     int n, fa[MXN], sz[MXN];
5     vector<pair<int*, int*>> h;
6     vector<int> sp;
7     void init(int tn) {
8         n = tn;
9         for (int i = 0; i < n; i++) sz[fa[i]] = i + 1;
10        sp.clear(); h.clear();
11    }
12    void assign(int *k, int v) {
13        h.PB({k, *k});
14        *k = v;
15    }
16    void save() { sp.PB(SZ(h)); }
17    void undo() {
18        assert(!sp.empty());
19        int last = sp.back(); sp.pop_back();
20        while (SZ(h) != last) {
21            auto x = h.back(); h.pop_back();
22            *x.F = x.S;
23        }
24    }
25    int f(int x) {
26        while (fa[x] != x) x = fa[x];
27        return x;
28    }
29    void uni(int x, int y) {
30        x = f(x); y = f(y);
31        if (x == y) return;
32        if (sz[x] < sz[y]) swap(x, y);
33        assign(&sz[x], sz[x] + sz[y]);
34        assign(&fa[y], x);
35    }
36 } djs;

```

4.2 segment tree range update (lazy propagation)

```

1 // segment tree
2 // range query & range modify
3 class SGT {
4     using value_t = int;
5     using node_t = pair<value_t, int>;
6
7     int n;
8     vector<node_t> t;
9     vector<optional<value_t>> lz;
10    // [ tv+1 : tv+2*(tm-tl) ) -> Left subtree
11    int left(int tv) { return tv + 1; }
12    int right(int tv, int tl, int tm) {
13        return tv + 2 * (tm - tl);
14    }
15    // ** differ from case to case **
16    // query is "max" and modify is "add" here
17    node_t merge(const node_t& x, const node_t& y) { // associative function
18        return max(x, y);
19    }
20    void update(int tv, int len, const value_t& x) {
21        if (!lz[tv]) lz[tv] = x;
22        else lz[tv] = lz[tv].value() + x;
23        t[tv].fi = t[tv].fi + x;
24    }
25    // *****
26    void build(const vector<value_t>& v, int tv, int tl, int tr) {
27        if (tr - tl > 1) {
28            int tm = (tl + tr) / 2;
29            build(v, left(tv), tl, tm);
30            build(v, right(tv, tl, tm), tm, tr);
31            t[tv] = merge(t[left(tv)], t[right(tv, tl, tm)]);
32        } else t[tv] = {v[tl], tl};
33    }
34    void push(int tv, int tl, int tr) { // lazy propagation
35        if (!lz[tv]) return;
36        int tm = (tl + tr) / 2;
37        update(left(tv), tm - tl, lz[tv].value());
38        update(right(tv, tl, tm), tr - tm, lz[tv].value());
39        lz[tv].reset();
40    }
41    void set(int p, const value_t& x, int tv, int tl, int tr) {
42        if (tr - tl > 1) {
43            push(tv, tl, tr);
44            int tm = (tl + tr) / 2;
45            if (p < tm) set(p, x, left(tv), tl, tm);
46            else set(p, x, right(tv, tl, tm), tm, tr);
47            t[tv] = merge(t[left(tv)], t[right(tv, tl, tm)]);
48        } else t[tv].fi = x;
49    }

```

```

50 }
51 void rmodify(int l, int r, const value_t& x, int tv, int tl, int tr) {
52     if (!(l == tl && r == tr)) {
53         push(tv, tl, tr);
54         int tm = (tl + tr) / 2;
55         if (r <= tm) rmodify(l, r, x, left(tv), tl, tm);
56         else if (l >= tm) rmodify(l, r, x, right(tv, tl, tm), tm, tr);
57         else rmodify(l, tm, x, left(tv), tl, tm), rmodify(tm, r, x, right(tv, tl, tm), tm, tr);
58         t[tv] = merge(t[left(tv)], t[right(tv, tl, tm)]);
59     } else update(tv, tr - tl, x);
60 }
61 node_t rquery(int l, int r, int tv, int tl, int tr) {
62     if (l == tl && r == tr) return t[tv];
63     push(tv, tl, tr);
64     int tm = (tl + tr) / 2;
65     if (r <= tm) return rquery(l, r, left(tv), tl, tm);
66     else if (l >= tm) return rquery(l, r, right(tv, tl, tm), tm, tr);
67     else return merge(rquery(l, tm, left(tv), tl, tm), rquery(tm, r, right(tv, tl, tm), tm, tr));
68 }
69 public:
70 explicit SGT(const vector<value_t>& v) : n{v.size()}, t(2 * n - 1, lz(2 * n - 1) { build(v, 0, 0, n); } } {}
71 void set(int p, const value_t& x) { set(p, x, 0, 0, n); }
72 void rmodify(int l, int r, const value_t& x) { rmodify(l, r, x, 0, 0, n); }
73 // [l:r)
74 node_t rquery(int l, int r) { return rquery(l, r, 0, 0, n); } // [l:r)
75 };
76
77 int main() {
78     vector<long long> a = {1, 5, 2, 4, 3};
79     SGT st(a);
80
81     auto [val, idx] = st.rquery(0, 5);
82     cout << "Initial max: " << val << " at " << idx << "\n"; // (5, 1)
83
84     st.rmodify(1, 4, 3); // add 3 to indices 1..3
85     tie(val, idx) = st.rquery(0, 5);
86     cout << "After add: " << val << " at " << idx << "\n"; // (8, 1)
87
88     st.set(2, 10); // set a[2] = 10
89     tie(val, idx) = st.rquery(0, 5);
90     cout << "After set: " << val << " at " << idx << "\n"; // (10, 2)
91 }

```


4.3 segment tree prefix sum lower bound

```

1 class SGT {
2     int n;
3     vector<long long> t;
4     int left(int tv) { return tv + 1; }
5     int right(int tv, int tl, int tm) {
6         return tv + 2 * (tm - tl); }
7     void modify(int p, long long x, int tv,
8                 int tl, int tr) {
9         if (tr - tl > 1) {
10             int tm{(tl + tr) / 2};
11             if (p < tm) modify(p, x, left(tv),
12                                tl, tm);
13             else modify(p, x, right(tv, tl,
14                                     tm), tm, tr);
15             t[tv] = t[left(tv)] + t[right(tv),
16                           tl, tm];
17         } else t[tv] = x;
18     }
19     long long query(int l, int r, int tv,
20                     int tl, int tr) {
21         if (l == tl && r == tr) return t[tv];
22         int tm{(tl + tr) / 2};
23         if (r <= tm) return query(l, r, left(tv),
24                                    tl, tm);
25         else if (l >= tm) return query(l, r, right(tv,
26                                                       tl, tm),
27                                         tm, tr);
28         else return query(l, tm, left(tv),
29                             tl, tm) +
30                             query(tm, r, right(tv,
31                                                  tl, tm),
32                                 tm, tr);
33     }
34 public:
35     explicit SGT(int _n : n[_n], t(2 * n -
36     1) {}
37     void modify(int p, long long x) { modify(
38         p, x, 0, 0, n); }
39     long long query(int l, int r) { return
40         query(l, r, 0, 0, n); }
41     int ps_lower_bound(long long ps) { //
42         prefix sum lower bound
43         if (ps > t[0]) return n;
44         int tv{0}, tl{0}, tr{n};
45         while (tr - tl > 1) {
46             int tm{(tl + tr) / 2};
47             if (t[left(tv)] >= ps) tv = left
48                 (tv), tr = tm;
49             else ps -= t[left(tv)], tv =
50                 right(tv, tl, tm), tl = tm;
51         }
52         return tl;
53     }
54 };

```

4.4 Fenwick tree (BIT)

```

1 // 1-based
2 struct Fenwick {

```

```

3     int n;
4     vector<int> bit;
5     Fenwick(int _n=0): n(_n), bit(n+1, 0) {}
6     void update(int idx, int val) {
7         for (; idx <= n; idx += idx & -idx)
8             bit[idx] += val;
9     }
10    int query(int idx) {
11        int res = 0;
12        for (; idx > 0; idx -= idx & -idx)
13            res += bit[idx];
14        return res;
15    }
16    int query(int l, int r) {
17        return query(r) - query(l-1);
18    }
19 };
20 int main() {
21     Fenwick fw(n);
22     for (int i = 1; i < n; ++i) {
23         fw.update(i, a[i]);
24     }
25     cout << fw.query(3, 7) << "\\n"; //
26         range sum [3..7]
27     int current = ...; // old value at idx
28     int newVal = ...; // new value you want
29     fw.update(idx, newVal - current);
30 }

```

4.5 Trie (Prefix tree)

```

1 struct trie {
2     int n = 0;
3     trie *a[2];
4     trie() {
5         a[0] = a[1] = nullptr;
6     }
7     void insert(int k) {
8         trie *curr = this;
9         for (int i = 63; i >= 0; --i) {
10             bool bit = (k & (1LL << i)) > 0;
11             if (curr->a[bit] == nullptr) {
12                 curr->a[bit] = new trie();
13             }
14             curr = curr->a[bit];
15             curr->n++;
16         }
17     }
18     void erase(int k) {
19         trie *curr = this;
20         for (int i = 63; i >= 0; --i) {
21             bool bit = (k & (1LL << i)) > 0;
22             curr = curr->a[bit];
23             curr->n--;
24         }
25     }
26     int query(int k) {
27         trie *curr = this;
28         int x = 0;

```

```

32     for (int i = 63; i >= 0; --i) {
33         x ^= (1LL << i) & k;
34         x ^= 1LL << i;
35         bool bit = (k & (1LL << i)) ==
36             0;
37         if (curr->a[bit] == nullptr ||
38             curr->a[bit]->n == 0)
39             x ^= 1LL << i, bit ^= 1;
40         curr = curr->a[bit];
41     }
42     return x;
43 };

```

4.6 segment tree

```

1 // Segment tree
2 template<typename value_t, class merge_t>
3 class SGT {
4     int n;
5     vector<value_t> t;
6     value_t defa;
7     merge_t merge;
8 public:
9     explicit SGT(int _n, value_t _defa,
10                  const merge_t& _merge = merge_t{})
11         : n(_n), t(2 * n), defa(_defa),
12           merge(_merge) {}
13     void modify(int p, const value_t& x) {
14         for (t[p += n] = x; p > 1; p >>= 1)
15             t[p >> 1] = merge(t[p], t[p ^
16                               1]);
17     }
18     value_t query(int l, int r) { return
19         query(l, r, defa); }
20     value_t query(int l, int r, value_t init)
21     {
22         for (l += n, r += n; l < r; l >>= 1,
23              r >>= 1) {
24             if (l & 1) init = merge(init, t[
25                 l++]);
26             if (r & 1) init = merge(init, t[
27                 --r]);
28         }
29         return init;
30     }
31 };
32 // Custom merge for range minimum + index
33 struct MergeMin {
34     pair<int, int> operator()(const pair<int,
35                               int>& a,
36                               const pair<int,
37                               int>& b) const {
38         if (a.first != b.first) return (a.
39             first < b.first) ? a : b;
40         return (a.second < b.second) ? a : b
41             ; // tie-break on index
42     }
43 };

```

4.7 BIT range update point query

```

1 // Fenwick Tree (Binary Indexed Tree) for
2     Range Updates and Point Queries
3 template<typename T>
4 class BIT {
5     #define ALL(x) begin(x), end(x)
6 private:
7     vector<T> arr;
8     int n;
9     inline int lowbit(int x) { return x & (-
10         x); }
11     void addInternal(int s, T v) {
12         while (s > 0) {
13             arr[s] += v;
14             s -= lowbit(s);
15         }
16 public:
17     void init(int n_) {
18         n = n_;
19         arr.resize(n + 1);
20         fill(ALL(arr), 0);
21     }
22     void add(int l, int r, T v) {
23         // add v to interval [l, r], 1-based
24         addInternal(l, -v);
25         addInternal(r, v);
26     }
27     T query(int x) {
28         // value at index x
29         T res = 0;
30         while (x <= n) {
31             res += arr[x];
32             x += lowbit(x);
33         }
34         return res;
35     }
36     #undef ALL
37 };
38 int main() {

```

```

40 BIT<int> bit;
41 bit.init(5);
42
43 // add +3 to indices 1..3
44 bit.add(0, 3, 3);
45
46 // add +2 to indices 3..5
47 bit.add(2, 5, 2);
48
49 cout << "Value at 1 = " << bit.query(1)
50 << "\n"; // expect 3
51 cout << "Value at 3 = " << bit.query(3)
52 << "\n"; // expect 3+2=5
53 cout << "Value at 5 = " << bit.query(5)
54 << "\n"; // expect 2
55 }

```

4.8 DSU remove node find prev next one

```

1 // previous/next one
2 class PvnX {
3     vector<int> pa, sz, mn, mx;
4     int find(int x) { // collapsing find
5         return pa[x] == -1 ? x : pa[x] =
6             find(pa[x]);
7     }
8     void unionn(int x, int y) { // weighted
9         union
10         auto rx{find(x)}, ry{find(y)};
11         if (rx == ry) return;
12         if (sz[rx] < sz[ry]) swap(rx, ry);
13         pa[ry] = rx, sz[rx] += sz[ry], mn[rx]
14         = min(mn[rx], mn[ry]), mx[rx]
15         = max(mx[rx], mx[ry]);
16     }
17 public:
18     explicit PvnX(int n) : pa(n + 1, -1), sz
19         (n + 1, 1), mn(n + 1) { iota(mn.
20         begin(), mn.end(), 0), mx = mn; }
21     void remove(int i) { unionn(i, i + 1); }
22     int prev(int i) { return mn[find(i)] -
23         1; }
24     int next(int i) {
25         int j{mx[find(i)]};
26         if (i == j) j = mx[find(j + 1)];
27         return j;
28     }
29     bool exist(int i) { return i == mx[find(
30         i)]; }
31 };

```

4.9 DSU

```

1 // fast disjoint set union
2 class DSU {
3     vector<int> pa, sz;
4 public:

```

```

5     explicit DSU(int n) : pa(n, -1), sz(n,
6         1) {}
7     int find(int x) { // collapsing find
8         return pa[x] == -1 ? x : pa[x] =
9             find(pa[x]);
10    }
11    void unite(int x, int y) { // weighted
12        union
13        auto rx{find(x)}, ry{find(y)};
14        if (rx == ry) return;
15        if (sz[rx] < sz[ry]) swap(rx, ry);
16        pa[ry] = rx, sz[rx] += sz[ry];
17    }
18 };

```

5 Geometry

5.1 cross. product

```

1 // If cross(a,b,c) > 0, c is to the left of
2 // line ab
3 // If cross(a,b,c) < 0, c is to the right of
4 // line ab
5 // If cross(a,b,c) = 0, a, b, c are
6 // collinear
7
8 // point struct version
9 // 2D Point structure
10 struct Point {
11     double x, y;
12 };
13
14 // Cross product (scalar in 2D)
15 double cross(const Point& a, const Point& b,
16     const Point& c) {
17     // Computes (b - a) x (c - a)
18     return (b.x - a.x) * (c.y - a.y) -
19         (b.y - a.y) * (c.x - a.x);
20 }
21
22 // std::complex version
23 typedef std::complex<double> point;
24 #define x real()
25 #define y imag()
26
27 double cross(const point &a, const point &b)
28 {
29     return (std::conj(a) * b).imag();
30 }

```

5.2 Check Point in Polygon

```

1 /*
2 We can cast a ray from the point P going in
3 any fixed direction (people usually go
4 to the right).

```

```

3 If the point is located on the outside of
4 the polygon the ray will intersect its
5 edges an even number of times.
6 If the point is on the inside of the polygon
7 then it will intersect the edge an odd
8 number of times.
9
10 */
11 struct Point {
12     int x, y;
13 };
14
15 int cross(const Point& a, const Point& b,
16     const Point& c) {
17     // return (c - a) x (c - b)
18     int ans = (c.x - a.x) * (c.y - b.y) - (c
19         .x - b.x) * (c.y - a.y);
20     if (ans == 0) return 0;
21     return ans > 0 ? 1 : -1;
22 }
23
24 bool intersect(const Point& a, const Point&
25     b, const Point& c, const Point& d) {
26     int c1 = cross(a, b, c);
27     int c2 = cross(a, b, d);
28     int c3 = cross(c, d, a);
29     int c4 = cross(c, d, b);
30
31     return (c1 * c2 < 0 && c3 * c4 < 0);
32 }

```

```

33 bool onSegment(const Point& a, const Point&
34     b, const Point& c) {
35     return (cross(a, b, c) == 0 &&
36         min(a.x, b.x) <= c.x && c.x <=
37         max(a.x, b.x) &&
38         min(a.y, b.y) <= c.y && c.y <=
39         max(a.y, b.y));
40 }
41
42 void solve() {
43     int n, m;
44     cin >> n >> m;
45     vector<Point> poly(n);
46
47     for (int i = 0; i < n; ++i) {
48         int a, b;
49         cin >> a >> b;
50         poly[i] = {a, b};
51     }
52
53     // set a point at outside point to form
54     the ray
55     Point outside = {(int)2e9 + 7, (int)2e9
56         + 13};
57     for (int i = 0; i < m; ++i) {
58         int a, b;
59         bool found = false;
60         cin >> a >> b;
61         Point p = {a, b};
62
63         int cnt = 0;
64         if (intersect(poly.back(), poly[0],
65             p, outside)) {
66             cnt++;
67         }
68     }

```

```

56     if (onSegment(poly.back(), poly[0],
57         p)) {
58         cout << "BOUNDARY\n";
59         found = true;
60     }
61     for (int j = 0; j < n - 1 && !found;
62         ++j) {
63         if (intersect(poly[j], poly[j +
64             1], p, outside)) {
65             cnt++;
66         }
67         if (onSegment(poly[j], poly[j +
68             1], p)) {
69             cout << "BOUNDARY\n";
70             found = true;
71         }
72     }
73     if (found) continue;
74     if (cnt & 1) {
75         cout << "INSIDE\n";
76     }
77     else {
78         cout << "OUTSIDE\n";
79     }
80 }

```

5.3 Point

```

1 /**
2 * Author: Ulf Lundstrom
3 * Date: 2009-02-26
4 * License: CC0
5 * Source: My head with inspiration from
6 tinyKACTL
7 * Description: Class to handle points in
8 the plane.
9 * T can be e.g. double or long long. (
10 Avoid int.)
11 * Status: Works fine, used a lot
12 */
13 #pragma once
14
15 template <class T> int sgn(T x) { return (x
16     > 0) - (x < 0); }
17 template <class T>
18 struct Point {
19     typedef Point P;
20     T x, y;
21     explicit Point(T x=0, T y=0) : x(x), y(y)
22     {}
23     bool operator<(P p) const { return tie(x, y)
24         < tie(p.x, p.y); }
25     bool operator==(P p) const { return tie(x, y)
26         == tie(p.x, p.y); }
27     P operator+(P p) const { return P(x+p.x, y
28         +p.y); }
29     P operator-(P p) const { return P(x-p.x, y
30         -p.y); }
31     P operator*(T d) const { return P(x*d, y*d)
32     }; }
33     P operator/(T d) const { return P(x/d, y/d)
34     }; }

```

```

24 T dot(P p) const { return x*p.x + y*p.y; }
25 T cross(P p) const { return x*p.y - y*p.x; }
26 T cross(P a, P b) const { return (a-*this)
    .cross(b-*this); }
27 T dist2() const { return x*x + y*y; }
28 double dist() const { return sqrt((double)
    dist2()); }
29 // angle to x-axis in interval [-pi, pi]
30 double angle() const { return atan2(y, x); }
31 P unit() const { return *this/dist(); } //
    makes dist()==1
32 P perp() const { return P(-y, x); } //
    rotates +90 degrees
33 P normal() const { return perp().unit(); }
34 // returns point rotated 'a' radians ccw
    around the origin
35 P rotate(double a) const {
36     return P(x*cos(a)-y*sin(a), x*sin(a)+y*
        cos(a)); }
37 friend ostream& operator<<(ostream& os, P
    p) {
38     return os << "(" << p.x << ", " << p.y <<
        ")"; }
39 };

```

5.4 Polygon Lattice Points

```

1 /*
2  Pick's Theorem:
3  For a simple polygon with integer
    coordinates, the area A, the number of
    interior lattice points a
4  and the number of boundary lattice points b
    satisfy:
5  A = a + b/2 - 1
6  */
7
8 /*
9  The number of intersections of a line with
    lattice points is the greatest common
    divisor of
10 P1.x - P2.x and P1.y - P2.y
11 */
12 int countOnSegment(const Point& a, const
    Point& b) {
13     int dx = abs(a.x - b.x);
14     int dy = abs(a.y - b.y);
15     return __gcd(dx, dy);
16 }
17
18 void solve() {
19     int n;
20     cin >> n;
21     vector<Point> poly(n);
22
23     for (int i = 0; i < n; ++i) {
24         cin >> poly[i].x >> poly[i].y;
25     }
26
27     // b: boundary points

```

```

28     int b = countOnSegment(poly.back(), poly
        [0]);
29     for (int i = 0; i < n - 1; ++i) {
30         b += countOnSegment(poly[i], poly[i
            + 1]);
31     }
32     int area2 = abs(polygonArea2(poly));
33     // a: interior points
34     int a = (area2 + 2 - b) / 2;
35     cout << a << " " << b << "\n";
36 }

```

5.5 line intersect

```

1 // 2D Point structure
2 struct Point {
3     double x, y;
4 };
5
6 // Cross product (scalar in 2D)
7 double cross(const Point& a, const Point& b,
    const Point& c) {
8     // Computes (b - a) x (c - a)
9     return (b.x - a.x) * (c.y - a.y) -
        (b.y - a.y) * (c.x - a.x);
10 }
11
12 // Check if value is between two others (
    with endpoints allowed)
13 bool between(double a, double b, double x) {
14     return min(a, b) <= x && x <= max(a, b);
15 }
16
17 // Check if point c lies on segment ab
18 bool onSegment(const Point& a, const Point&
    b, const Point& c) {
19     return cross(a, b, c) == 0 &&
        between(a.x, b.x, c.x) &&
        between(a.y, b.y, c.y);
20 }
21
22 // Main intersection check
23 bool segmentsIntersect(const Point& A, const
    Point& B, const Point& C, const
    Point& D) {
24     double c1 = cross(A, B, C);
25     double c2 = cross(A, B, D);
26     double c3 = cross(C, D, A);
27     double c4 = cross(C, D, B);
28
29     // General case
30     if (c1 * c2 < 0 && c3 * c4 < 0) return
        true;
31
32     // Special cases: collinear +
        overlapping
33     if (c1 == 0 && onSegment(A, B, C))
34         return true;
35     if (c2 == 0 && onSegment(A, B, D))
36         return true;
37     if (c3 == 0 && onSegment(C, D, A))
38         return true;
39     if (c4 == 0 && onSegment(C, D, B))
40         return true;

```

```

41     if (c4 == 0 && onSegment(C, D, B))
42         return true;
43     return false;
44 }
45
46 int main() {
47     Point A{0, 0}, B{4, 4};
48     Point C{0, 4}, D{4, 0};
49
50     if (segmentsIntersect(A, B, C, D))
51         cout << "Segments intersect\n";
52     else
53         cout << "Segments do not intersect\n";
54
55     return 0;
56 }

```

5.6 Polygon area

```

1 /**
2  * Author: Ulf Lundstrom
3  * Date: 2009-03-21
4  * License: CC0
5  * Source: tinyKACTL
6  * Description: Returns twice the signed
    area of a polygon.
7  * Clockwise enumeration gives negative
    area. Watch out for overflow if using
    int as T!
8  * Status: Stress-tested and tested on
    kattis:polygonarea
9  */
10 #pragma once
11
12 #include "Point.h"
13
14 template<class T>
15 T polygonArea2(vector<Point<T>>& v) {
16     T a = v.back().cross(v[0]);
17     for (int i = 0; i < (int)v.size() - 1;
        ++i) {
18         a += v[i].cross(v[i+1]);
19     }
20     return a;
21 }
22
23 int main() {
24     // Example: square with vertices (0,0),
        (0,1), (1,1), (1,0)
25     vector<Point<long long>> poly = {
26         {0,0}, {0,1}, {1,1}, {1,0}
27     };
28
29     long long area2 = polygonArea2(poly);
30     cout << "Twice signed area = " << area2
        << "\n";
31     cout << "Absolute area = " << abs(area2)
        / 2.0 << "\n";
32
33     return 0;
34 }

```

5.7 using std::complex

```

1 #include <iostream>
2 #include <complex>
3 #include <cmath>
4
5 typedef std::complex<double> point;
6 #define x real()
7 #define y imag()
8
9 constexpr double PI = std::acos(-1.0);
10 // conj(a) = a 的共軛 · reflection of a
    across x-axis
11
12 // Basic operations
13 double dot(const point &a, const point &b) {
14     return (std::conj(a) * b).real(); }
15
16 double cross(const point &a, const point &b)
    { return (std::conj(a) * b).imag(); }
17
18 // Projections / reflections / geometry
    helpers
19 point project_onto_vector(const point &p,
    const point &v) {
20     return v * (dot(p, v) / std::norm(v));
21 }
22
23 point project_onto_line(const point &p,
    const point &a, const point &b) {
24     return a + (b - a) * (dot(p - a, b - a)
        / std::norm(b - a));
25 }
26
27 point reflect_across_line(const point &p,
    const point &a, const point &b) {
28     return a + std::conj((p - a) / (b - a))
        * (b - a);
29 }
30
31 point intersection(const point &a, const
    point &b, const point &p, const point &q)
    {
32     double c1 = cross(p - a, b - a), c2 =
        cross(q - a, b - a);
33     return (c1 * q - c2 * p) / (c1 - c2); //
        undefined if parallel (divide by
        zero)
34 }
35
36 double angle_between(const point &a, const
    point &b) {
37     // angle of vector b - a
38     return std::arg(b - a);
39 }
40
41 double angle_ABC(const point &a, const point
    &b, const point &c) {
42     double r = std::remainder(std::arg(a - b)
        - std::arg(c - b), 2.0 * PI);
43     return std::abs(r);
44 }
45
46 int main() {
47     // sample
48     point a = 2.0; // (2,0)

```



```

47 point b(3.0, 7.0); // (3,7)
48 std::cout << a << ' ' << b << '\n'; //
49 (2,0) (3,7)
50 std::cout << a + b << '\n'; //
51 (5,7)
52 // usage examples
53 point p1(3, 2), p2(2, -7);
54 std::cout << "p1 + p2 = " << p1 + p2 <<
55 '\n'; // (5,-5)
56 std::cout << "p1 - p2 = " << p1 - p2 <<
57 '\n'; // (1,9)
58 std::cout << "3.0 * p1 = " << 3.0 * p1
59 << '\n'; // (9,6)
60 std::cout << "p1 / 5.0 = " << p1 / 5.0
61 << '\n'; // (0.6,0.4)
62 // dot / cross via complex
63 std::cout << "dot(p1,p2) = " << dot(p1,
64 p2) << '\n';
65 std::cout << "cross(p1,p2) = " << cross(
66 p1, p2) << '\n';
67 // distances, angle, rotation, polar/
68 cartesian
69 std::cout << "squared dist = " << std::
70 norm(p1 - p2) << '\n';
71 std::cout << "euclid dist = " << std::
72 abs(p1 - p2) << '\n';
73 std::cout << "angle p1->p2 = " <<
74 angle_between(p1, p2) << '\n';
75 std::cout << "angle ABC = " << angle_ABC
76 (point(1,0), point(0,0), point(0,1))
77 << '\n';
78 // project / reflect / intersect
79 examples
80 point v(1, 1);
81 point proj = project_onto_vector(p1, v);
82 std::cout << "project p1 onto v = " <<
83 proj << '\n';
84 point lineA(0,0), lineB(2,0);
85 std::cout << "project p1 onto line = "
86 << project_onto_line(p1, lineA,
87 lineB) << '\n';
88 std::cout << "reflect p1 across line = "
89 << reflect_across_line(p1, lineA,
90 lineB) << '\n';
91 // intersection example
92 point r1a(0,0), r1b(1,1);
93 point r2a(0,1), r2b(1,0);
94 std::cout << "intersection = " <<
95 intersection(r1a, r1b, r2a, r2b) <<
96 '\n';
97 // polar/cartesian and rotation
98 point polar_pt = std::polar(5.0, PI/4);
99 std::cout << "polar(5,PI/4) = " <<
100 polar_pt << '\n';
101 point rotated = p1 * std::polar(1.0, PI
102 /2); // rotate p1 by 90 degrees
103 about origin
104 std::cout << "rotate p1 by 90deg = " <<
105 rotated << '\n';

```

5.8 point distance from a line

```

1 // calculate area using cross product and
2 divide by base length
3 double point_line_distance(const point &p,
4 const point &a, const point &b) {
5 // area = 0.5 * |cross(b - a, p - a)|
6 // base = |b - a|
7 // height = 2 * area / base = |cross(b -
8 a, p - a)| / |b - a|
9 std::abs(cross(b - a, p - a)) / std::abs
10 (b - a);
11 }

```

6 Graph

6.1 Euler tour+RMQ

```

1 // Euler Tour Technique
2 class LCA {
3     const vector<vector<int>>& adj;
4     int n;
5     vector<int> d, first, euler{}, log2{};
6     vector<vector<int>> st{};
7     void dfs(int u, int w = -1, int dep = 0)
8     {
9         d[u] = dep;
10        first[u] = euler.size();
11        euler.push_back(u);
12        for (auto& v : adj[u]) {
13            if (v == w) continue;
14            dfs(v, u, dep + 1);
15            euler.push_back(u);
16        }
17    }
18 public:
19     LCA(const vector<vector<int>>& _adj, int
20         root) : adj{ _adj }, n{ adj.size() }, d
21         (n), first(n) {
22         dfs(root);
23     }
24     int tn{euler.size()};
25     log2.resize(tn + 1);
26     log2[1] = 0;
27     for (int i{2}; i <= tn; ++i) log2[i]
28         = log2[i / 2] + 1;
29     st.assign(tn, vector<int>(log2[tn] +
30         1));
31     for (int i{tn - 1}; i >= 0; --i) {
32         st[i][0] = euler[i];
33         for (int j{1}; i + (1 << j) <=
34             tn; ++j) {
35             auto& x{st[i][j - 1]};

```

```

87 return 0;
88 }
89 }

```

```

31 auto& y{st[i + (1 << (j - 1)
32 )][j - 1]};
33 st[i][j] = d[x] <= d[y] ? x
34 : y;
35 }
36 }
37 int operator()(int u, int v) {
38     int l{first[u]}, r{first[v]};
39     if (l > r) swap(l, r);
40     ++r; // make the interval left
41         closed right open
42     int j{log2[r - l]};
43     auto& x{st[l][j]};
44     auto& y{st[r - (1 << j)][j]};
45     return d[x] <= d[y] ? x : y;
46 }
47 };
48 int main() {
49     int n, q;
50     cin >> n >> q;
51     vector<vector<int>> adj(n);
52     for (int i = 1; i < n; ++i) {
53         int u;
54         cin >> u;
55         u--;
56         adj[u].pb(i);
57         adj[i].pb(u);
58     }
59     LCA lca(adj, 0);
60     while (q--) {
61         int u, v;
62         cin >> u >> v;
63         u--, v--;
64         cout << lca(u, v) + 1 << "\n";
65     }
66     return 0;
67 }

```

6.2 Prim

```

1 // Prim's algorithm
2 vector<tuple<int, int, long long>> Prim(
3     const vector<vector<pair<int, long long>
4     >>& adj) {
5     const auto& n = adj.size();
6     vector<tuple<int, int, long long>> mst
7     {};
8     vector<bool> found(n, false);
9     using ti = tuple<long long, int, int>;
10    priority_queue<ti, vector<ti>, greater<
11        ti>> pq{};
12    found[0] = true;
13    for (auto& [v, w] : adj[0]) pq.emplace(w
14        , 0, v);
15    for (int i = 0; i < n - 1; ++i) {

```

```

13 int mn, u, v;
14 do {
15     tie(mn, u, v) = pq.top(), pq.pop()
16     ();
17 } while (found[v]);
18 found[v] = true, mst.emplace_back(u,
19     v, mn);
20 for (auto& [x, w] : adj[v]) pq.
21     emplace(w, v, x);
22 }
23 return mst;
24 }

```

6.3 Eulerian cycle

```

1 // Eulerian cycle in an undirected graph
2 vector<int> euler_cycle(vector<vector<pair<
3     int, int>>& adj, int w = 0) {
4     int n{adj.size()};
5     for (int v{0}; v < n; ++v) m += adj[v].
6     size();
7     m /= 2;
8     vector<int> res{};
9     stack<pair<int, int>> stk{};
10    stk.emplace(w, -1);
11    vector<int> nxt(n);
12    vector<bool> used(m);
13    while (!stk.empty()) {
14        auto [u, i]{stk.top()};
15        while (nxt[u] < adj[u].size() && used
16            [adj[u][nxt[u]].second]) ++nxt[u]
17            ;
18        if (nxt[u] < adj[u].size()) {
19            auto [v, j]{adj[u][nxt[u]]};
20            ++nxt[u], used[j] = true, stk.
21                emplace(v, j);
22        } else {
23            if (i != -1) res.push_back(i);
24            stk.pop();
25        }
26    }
27    return res;
28 }
29 int main() {
30     int n = 4; // number of vertices
31     vector<vector<pair<int, int>>> adj(n);
32     // Add edges with edge IDs
33     int eid = 0;
34     auto add_edge = [&](int u, int v) {
35         adj[u].push_back({v, eid});
36         adj[v].push_back({u, eid});
37         ++eid;
38     };
39     add_edge(0, 1);
40     add_edge(1, 2);
41     add_edge(2, 3);
42     add_edge(3, 0);
43 }

```

```

44 vector<int> cycle = euler_cycle(adj, 0);
45 cout << "Euler cycle (edge IDs in order)
46 : ";
47 for (int id : cycle) cout << id << " ";
48 cout << "\n";
49
50 return 0;
51 }

```

6.4 maximum weight bipartite matching

```

1 // maximum weight matching in a weighted
  bipartite graph
2 // Hungarian Algorithm -  $O(V^3)$ 
3 class bipartiteGraph {
4     int n;
5     vector<long long> lx, ly;
6     vector<bool> vx, vy;
7     queue<int> qu; // only X's vertices
8     vector<int> py;
9     vector<long long> dy; // smallest (lx[x]
    + ly[y] - w[x][y])
10    vector<int> pdy; // & which x
11    void relax(int x) {
12        for (int y{0}; y < n; ++y)
13            if (lx[x] + ly[y] - w[x][y] < dy
                [y])
14                dy[y] = lx[x] + ly[y] - w[x]
                    [y], pdy[y] = x;
15    }
16    void augment(int y) {
17        while (y != -1) {
18            int x{py[y]}, yy{mx[x]};
19            mx[x] = y, my[y] = x;
20            y = yy;
21        }
22    }
23    bool bfs() {
24        while (!qu.empty()) {
25            int x{qu.front()}; qu.pop();
26            for (int y{0}; y < n; ++y) {
27                if (!vy[y] && lx[x] + ly[y]
                    == w[x][y]) {
28                    vy[y] = true, py[y] = x;
29                    if (my[y] == -1) return
                        augment(y), true;
30                    int xx{my[y]};
31                    qu.push(x), vx[xx] =
                        true, relax(xx);
32                }
33            }
34        }
35        return false;
36    }
37    void relabel() {
38        long long d{numeric_limits<long long>
            ::max()};
39        for (int y{0}; y < n; ++y) if (!vy[y]
            ) d = min(d, dy[y]);

```

```

40        for (int x{0}; x < n; ++x) if (vx[x]
            ) lx[x] -= d;
41        for (int y{0}; y < n; ++y) if (vy[y]
            ) ly[y] += d;
42        for (int y{0}; y < n; ++y) if (!vy[y]
            ) dy[y] -= d;
43    }
44    bool expand() { // expand one layer with
    new equality edges
45        for (int y{0}; y < n; ++y) {
46            if (!vy[y] && dy[y] == 0) {
47                vy[y] = true, py[y] = pdy[y]
                    ;
48                if (my[y] == -1) return
                    augment(y), true;
49                int xx{my[y]};
50                qu.push(xx), vx[xx] = true,
                    relax(xx);
51            }
52        }
53        return false;
54    }
55    public:
56    vector<vector<long long>> w;
57    vector<int> mx, my;
58    bipartiteGraph(int _n : n[_n], lx(n),
        ly(n), vx(n), vy(n), py(n), dy(n),
        pdy(n),
59        w(n, vector<long long>(n, 0)), mx(n,
        -1), my(n, -1) {}
60    long long Hungarian() {
61        for (int i{0}; i < n; ++i) {
62            lx[i] = ly[i] = 0;
63            for (int j{0}; j < n; ++j)
64                lx[i] = max(lx[i], w[i][j]);
65            mx[i] = my[i] = -1;
66        }
67        for (int x{0}; x < n; ++x) {
68            fill(vx.begin(), vx.end(), false
                );
69            fill(vy.begin(), vy.end(), false
                );
70            fill(dy.begin(), dy.end(),
                numeric_limits<long long>::
                    max());
71            qu = {}, qu.push(x), vx[x] =
                true, relax(x);
72            while (!bfs()) {
73                relabel();
74                if (expand()) break;
75            }
76        }
77        long long weight{0};
78        for (int x{0}; x < n; ++x) weight +=
            w[x][mx[x]];
79        return weight;
80    }
81    };
82
83 // usage example:
84 int main() {
85     int n = 4;
86     bipartiteGraph g(n);

```

```

35 long long mat[4][4] = {
36     {9, 2, 7, 8},
37     {6, 4, 3, 7},
38     {5, 8, 1, 8},
39     {7, 6, 9, 4}
40 };
41 for (int i = 0; i < n; ++i)
42     for (int j = 0; j < n; ++j)
43         g.w[i][j] = mat[i][j];
44
45 long long maxWeight = g.Hungarian();
46 std::cout << "Maximum weight: " <<
    maxWeight << "\n";
47 for (int i = 0; i < n; ++i)
48     std::cout << "X " << i << " matched
        with Y " << g.mx[i] << "\n";
49 }

```

6.5 maximum flow

```

1 // minimum cut maximum flow
2 // relabel-to-front algorithm
3 // (an implementation of generic push-relabel
    algorithm)
4
5 // handle the capacity, c, of a vertex, v.
6 // Add a new vertex, v', and an edge v-v'
    with capacity c.
7 template<typename T> //requires
    signed_integral<T>
8 void handle_vertex_capacity(int u, T c, int&
    n, vector<tuple<int, int, T>>& e) {
9     int w{n++};
10    for (auto& [_u, _v, _c] : e) if (_u == u
        ) _u = w;
11    e.emplace_back(u, w, c);
12 }
13
14 template<typename T> //requires
    signed_integral<T>
15 class flowNetwork {
16     int n, s, t;
17     vector<int> h; // height label
18     vector<T> e; // excess
19     vector<vector<int>>> adj; // adjacent
        list
20     vector<vector<T>>> c, r;
21     vector<size_t> p; // record the position
        processed
22     list<int> lst; // topological sort of
        admissible network
23     void push(int u, int v) {
24         T f{min(e[u], r[u][v])};
25         r[u][v] -= f, r[v][u] += f;
26         e[u] -= f, e[v] += f;
27     }
28     void relabel(int u) {
29         int mn{numeric_limits<int>::max()};
30         for (auto& v : adj[u])
31             if (r[u][v]) mn = min(mn, h[v]);
32         h[u] = mn + 1;
33     }
34     void init() {

```

```

35     fill(h.begin(), h.end(), 0);
36     fill(e.begin(), e.end(), 0);
37     lst.clear();
38     for (int u{0}; u < n; ++u) if (u !=
        s && u != t) lst.push_back(u);
39     fill(p.begin(), p.end(), 0);
40     r = c;
41 }
42 void preflow() {
43     h[s] = n;
44     for (auto& v : adj[s]) {
45         e[v] += r[s][v], e[s] -= r[s][v]
            ;
46         r[v][s] += r[s][v], r[s][v] = 0;
47     }
48 }
49 bool discharge(int u) {
50     bool flag{false};
51     while (e[u]) {
52         for (; p[u] < adj[u].size(); ++p
            [u]) {
53             int v{adj[u][p[u]]};
54             if (r[u][v] && h[u] == h[v]
                + 1) push(u, v);
55             if (!e[u]) break;
56         }
57         if (e[u]) relabel(u), p[u] = 0,
            flag = true;
58     }
59     return flag; // return if relabel or
        not
60 }
61 public:
62 flowNetwork(int _n : n[_n], s{0}, t{n -
    1}, h(n), e(n), adj(n), c(n, vector
        <T>(n)), p(n) {}
63 void set_s(int _s, int _t) {
64     s = _s, t = _t;
65 }
66 void add_edge(int u, int v, T _c) {
67     if (!c[u][v] && !c[v][u]) adj[u].
        push_back(v), adj[v].push_back(u
            );
68     c[u][v] += _c;
69 }
70 T max_flow() {
71     assert(s != t);
72
73     init();
74     preflow();
75
76     auto it{lst.begin()};
77     while (it != lst.end()) {
78         if (discharge(*it)) { // relabel
            -to-front
79             lst.push_front(*it), lst.
                erase(it);
80             it = lst.begin();
81         }
82         it = next(it);
83     }
84     return e[t];
85 };
86
87 };
88

```

```

89 int main() {
90     using T = int;
91
92     // Build an example graph with a vertex
93     // capacity on node 2.
94     int n = 4; // nodes: 0 (source), 1, 2, 3
95     // (sink)
96     vector<tuple<int, int, T>> edges = {
97         {0, 1, 3},
98         {0, 2, 2},
99         {1, 2, 5},
100         {1, 3, 2},
101         {2, 3, 3}
102     };
103
104     // Enforce vertex capacity of 2 on node
105     // 2.
106     handle_vertex_capacity<T>(2, 2, n, edges
107     );
108
109     // Build the network and compute max
110     // flow.
111     flowNetwork<T> net(n);
112     net.set_st(0, 3);
113     for (auto [u, v, cap] : edges) net.
114         add_edge(u, v, cap);
115
116     cout << "Max flow = " << net.max_flow()
117         << '\n';
118 }

```

6.6 maximum cardinality bipartite matching

```

1 class BipartiteMatching {
2     vector<bool> vis;
3     vector<int> mx{}, my{};
4     bool dfs(const vector<vector<int>>& adj,
5             int x) {
6         for (int y : adj[x]) {
7             if (vis[y]) continue;
8             vis[y] = true;
9             if (my[y] == -1 || dfs(adj, my[y]
10             )) {
11                 mx[x] = y; my[y] = x;
12                 return true;
13             }
14         }
15         return false;
16     }
17 public:
18     pair<vector<int>, vector<int>> operator
19     ()(const vector<vector<int>>& adj,
20         int ny) {
21         vis.assign(ny, false);
22         mx.assign((int)adj.size(), -1);
23         my.assign(ny, -1);
24         for (int x = 0; x < (int)adj.size();
25             ++x) {
26             fill(vis.begin(), vis.end(),
27                 false);

```

```

22         dfs(adj, x);
23     }
24     return {mx, my};
25 }
26 };
27
28 // Usage example:
29 int main() {
30     // Left partition has 4 nodes (0..3),
31     // right partition has 4 nodes (0..3).
32     vector<vector<int>> adj(4);
33     adj[0] = {0, 1};
34     adj[1] = {1, 2};
35     adj[2] = {2};
36     adj[3] = {2, 3};
37
38     BipartiteMatching bm;
39     auto res = bm(adj, 4);
40     const auto& mx = res.first;
41
42     cout << "Matched pairs (Left -> right):\n";
43     for (int i = 0; i < (int)mx.size(); ++i)
44         if (mx[i] != -1)
45             cout << i << " -> " << mx[i] <<
46                 "\n";
47 }

```

6.7 Floyd-Warshall

```

1 // Floyd-Warshall algorithm
2 template<typename T>
3 vector<vector<optional<T>>> Floyd_Warshall(
4     const vector<vector<optional<T>>>& adj)
5 {
6     const auto& n{adj.size()};
7     auto d{adj};
8
9     for (int i{0}; i < n; ++i) d[i][i] = 0;
10    for (int k{0}; k < n; ++k)
11        for (int i{0}; i < n; ++i)
12            for (int j{0}; j < n; ++j) {
13                if (!d[i][k] || !d[k][j])
14                    continue; // no value
15                if (!d[i][j] || d[i][j] > d[
16                    i][k].value() + d[k][j].
17                    value())
18                    d[i][j] = d[i][k].value
19                        () + d[k][j].value()
20                        ;
21            }
22    }
23    return d;
24 }

```

6.8 MST

```

1 // Kruskal' s algorithm

```

```

2 vector<tuple<int, int, long long>> Kruskal(
3     int n, vector<tuple<long long, int, int>
4     >>& edges) {
5     vector<tuple<int, int, long long>> mst
6     {};
7     DSU dsu{n};
8
9     sort(edges.begin(), edges.end());
10    for (auto& [w, u, v] : edges)
11        if (dsu.find(u) != dsu.find(v))
12            dsu.unionn(u, v), mst.
13            emplace_back(u, v, w);
14    return mst;
15 }

```

6.9 Hopcroft-Karp

```

1 // maximum cardinality bipartite matching in
2 // unweighted bipartite graph
3 // Hopcroft-Karp algorithm - O(EV)
4 class bipartiteGraph {
5     int nx, ny;
6     vector<int> dx, dy;
7     bool dfs(int y) {
8         for (auto& x : adjy[y]) {
9             if (dx[x] + 1 != dy[y]) continue
10             ;
11             if (mx[x] == -1 || dfs(mx[x])) {
12                 my[y] = x, mx[x] = y;
13                 dy[y] = -1; // augmented -
14                 // remove from BFS forest
15                 return true;
16             }
17         }
18         dy[y] = -1; // no augmenting path -
19         // remove from BFS forest
20         return false;
21     }
22     bool bfs() {
23         fill(dx.begin(), dx.end(), -1);
24         fill(dy.begin(), dy.end(), -1);
25         queue<int> qu{}; qu2{};
26         for (int x{0}; x < nx; ++x) // use
27             all unmatched x's as roots
28             if (mx[x] == -1) qu.push(x), dx[
29                 x] = 0;
30
31         bool found{false};
32         while (!qu.empty() && !found) { //
33             stop at the level found
34             while (!qu.empty()) {
35                 auto x{qu.front()}; qu.pop()
36                 ;
37                 for (auto& y : adjx[x])
38                     if (dy[y] == -1 && my[y]
39                         != x) {
40                         dy[y] = dx[x] + 1;
41                         if (my[y] == -1)
42                             found = true;
43                         else qu2.push(my[y])
44                             , dx[my[y]] = dy
45                             [y] + 1;
46                     }
47             }
48         }
49         return found;
50     }
51 }

```

```

35     }
36     qu.swap(qu2);
37 }
38 return found;
39 }
40 vector<vector<int>> adjx, adjy;
41 public:
42 vector<int> mx, my;
43 bipartiteGraph(int _nx, int _ny) : nx{
44     _nx}, ny{_ny}, dx(nx), dy(ny)
45     , adjx(nx), adjy(ny), mx(nx, -1), my
46     (ny, -1) {}
47 void addEdge(int x, int y) {
48     adjx[x].push_back(y), adjy[y].
49     push_back(x);
50 }
51 int Hopcroft_Karp() {
52     int c{0};
53     while (bfs()) {
54         for (int y{0}; y < ny; ++y)
55             if (my[y] == -1 && dy[y] !=
56                 -1) c += dfs(y);
57     }
58     return c;
59 }
60 // Usage example:
61 int main() {
62     // Left partition has 4 nodes (0..3),
63     // right partition has 4 nodes (0..3).
64     bipartiteGraph bg(4, 4);
65     bg.addEdge(0, 0);
66     bg.addEdge(0, 1);
67     bg.addEdge(1, 1);
68     bg.addEdge(1, 2);
69     bg.addEdge(2, 2);
70     bg.addEdge(3, 2);
71     bg.addEdge(3, 3);
72
73     int max_matching = bg.Hopcroft_Karp();
74     cout << "Maximum matching size: " <<
75         max_matching << "\n";
76     cout << "Matched pairs (Left -> right):\n";
77     for (int i = 0; i < (int)bg.mx.size();
78         ++i)
79         if (bg.mx[i] != -1)
80             cout << i << " -> " << bg.mx[i]
81                 << "\n";
82 }

```

6.10 all longest path dfs

```

1 // all longest path (generalization of the
2 // tree diameter problem)
3 vector<tuple<int, int, int>> dp{};
4 // [mx1, x, mx2] the path of mx1 goes
5 // through x
6 int dfs1(int u, int w = -1) {
7     int mx{0};
8     for (auto& v : adj[u])
9         if (v != w) {

```

```

8      auto l{1 + dfs1(v, u)};
9      mx = max(mx, l);
10
11      auto& [mx1, x, mx2]{dp[u]};
12      if (l >= mx1) mx2 = mx1, mx1 = l
13          , x = v;
14      else if (l > mx2) mx2 = l;
15  }
16  return mx;
17 void dfs2(int u, int w = -1) {
18     if (w != -1) {
19         int tmx;
20         { // calculate the longest path
21             through parent
22             auto& [mx1, x, mx2]{dp[w]};
23             if (x != u) tmx = mx1 + 1;
24             else tmx = mx2 + 1;
25         }
26         { // update the path
27             auto& [mx1, x, mx2]{dp[u]};
28             if (tmx >= mx1) mx2 = mx1, mx1 =
29                 tmx, x = w;
30             else if (tmx > mx2) mx2 = tmx;
31         }
32     }
33     for (auto& v : adj[u])
34         if (v != w) dfs2(v, u);
35 }
36 void all_longest_path() {
37     dfs1(0), dfs2(0);
38 }

```

6.11 all longest path top sort

```

1 // all longest path in DAG
2 // 1. topological sort
3 vector<int> in(n, 0);
4 for (int i = 0; i < m; ++i) {
5     int a, b, w;
6     cin >> a >> b >> w;
7     adj[a].emplace_back(b, w);
8     in[b]++;
9 }
10 vector<int> topo; // sequence of top sort
11 queue<int> q;
12 for (int i = 0; i < n; ++i) {
13     if (in[i] == 0) {
14         q.push(i);
15     }
16 }
17 while (!q.empty()) {
18     int pa = q.front();
19     q.pop();
20     topo.push_back(pa);
21     for (auto& [child, w] : adj[pa]) {
22         in[child]--;
23         if (in[child] == 0) {
24             q.push(child);
25         }
26     }
27 }
28 }

```

```

29 // all longest path
30 vector<int> dist(n, INT_MIN);
31 vector<vector<int>> parents(n);
32 dist[0] = process[0];
33 for (int u : topo) {
34     for (auto& [v, w] : adj[u]) {
35         if (dist[v] < dist[u] + process[v] +
36             w) {
37             dist[v] = dist[u] + process[v] +
38                 w;
39             parents[v] = {u};
40         }
41         else if (dist[v] == dist[u] +
42             process[v] + w) {
43             parents[v].push_back(u);
44         }
45     }
46 }
47 cout << dist[n - 1];

```

6.12 Dijkstra

```

1 // Dijkstra algorithm
2 template<typename T>
3 vector<optional<T>> Dijkstra(const vector<
4     vector<pair<int, T>>& adj, int s) {
5     const auto& n{adj.size()};
6     vector<optional<T>> d(n, nullopt);
7     d[s] = 0;
8
9     vector<bool> found(n, false);
10    using pq_t = pair<T, int>;
11    priority_queue<pq_t, vector<pq_t>,
12        greater<pq_t>> pq{};
13    pq.emplace(0, s);
14    while (!pq.empty()) {
15        auto [_, u]{pq.top()}; pq.pop();
16        if (found[u]) continue;
17        found[u] = true;
18        for (auto& [v, w] : adj[u])
19            if (!d[v] || d[v] > d[u].value()
20                + w) {
21                d[v] = d[u].value() + w;
22                pq.emplace(d[v].value(), v);
23            }
24    }
25    return d;
26 }

```

6.13 Heavy-light decomposition (HLD)

```

1 const int maxn = 2e5;
2 int n, q;
3 int val[maxn];

```

```

4 int id[maxn];
5 int sz[maxn];
6 int parent[maxn];
7 int depth[maxn];
8 int tp[maxn];
9 vector<int> adj[maxn];
10 int timer = 0;
11
12 void dfs_sz(int u, int p) {
13     sz[u] = 1;
14     parent[u] = p;
15
16     // calculate size of each subtree
17     for (int v : adj[u]) {
18         if (v != p) {
19             depth[v] = depth[u] + 1;
20             sz[u] += dfs_sz(v, u);
21         }
22     }
23     return sz[u];
24 }
25
26 void dfs_hld(int u, int p, int top, SGT<int,
27     MergeMax>& tree) {
28     id[u] = timer++;
29     tp[u] = top;
30     tree.update(id[u], val[u]);
31
32     // find the heavy child
33     int h_v = -1, h_sz = -1;
34     for (int v : adj[u]) {
35         if (v != p) {
36             if (h_sz < sz[v]) {
37                 h_sz = sz[v];
38                 h_v = v;
39             }
40         }
41     }
42     if (h_v == -1) {
43         return;
44     }
45     // continue the chain
46     dfs_hld(h_v, u, top, tree);
47     // start a new chain for each light
48     // child
49     for (int v : adj[u]) {
50         if (v != p && v != h_v) {
51             dfs_hld(v, u, v, tree);
52         }
53     }
54 }
55
56 int path(int x, int y, SGT<int, MergeMax>&
57     tree) {
58     int ans = 0;
59     // move x and y to their top nodes until
60     // they are in the same chain
61     while (tp[x] != tp[y]) {
62         if (depth[tp[x]] < depth[tp[y]])
63             swap(x, y);
64         ans = max(ans, tree.query(id[tp[x]],
65             id[x] + 1));
66         x = parent[tp[x]];
67     }
68     if (depth[x] > depth[y]) swap(x, y);
69 }

```

```

64 ans = max(ans, tree.query(id[x], id[y] +
65     1));
66 return ans;
67 }
68
69 void solve() {
70     cin >> n >> q;
71     SGT<int, MergeMax> tree(n, -1);
72
73     for (int i = 0; i < n; ++i) {
74         cin >> val[i];
75     }
76     for (int i = 0; i < n - 1; ++i) {
77         int a, b;
78         cin >> a >> b;
79         a--, b--;
80         adj[a].push_back(b);
81         adj[b].push_back(a);
82     }
83     dfs_sz(0, 0);
84     dfs_hld(0, 0, 0, tree);
85
86     cerr << "tp: ";
87     for (int i = 0; i < n; ++i) {
88         cerr << tp[i] << " ";
89     }
90     cerr << "\n";
91
92     while (q--) {
93         int mode;
94         cin >> mode;
95         if (mode == 1) {
96             int s, x;
97             cin >> s >> x;
98             s--;
99             val[s] = x;
100             tree.update(id[s], val[s]);
101         }
102         else {
103             int a, b;
104             cin >> a >> b;
105             cout << path(a - 1, b - 1, tree)
106                 << " ";
107         }
108     }
109 }

```

6.14 binary lifting

```

1 // binary lifting
2 class LCA {
3     const vector<vector<int>>& adj;
4     int n;
5     vector<int> d;
6     vector<int> log2;
7     vector<vector<int>> an{};
8     void dfs(int u, int w = -1, int dep = 0) {
9         d[u] = dep;
10        an[u][0] = w;
11        for (int i{1}; i <= log2[n - 1] &&
12            an[u][i - 1] != -1; ++i)

```

```

12   an[u][i] = an[an[u][i - 1]][i - 1]; // 走 2^(i-1) 再走 2^(i-1) 步
13
14   // 因為計算 an 會用到祖先的資訊，所以先計算再繼續往下
15   for (auto& v : adj[u]) {
16       if (v == w) continue; // parent
17       dfs(v, u, dep + 1);
18   }
19
20 public:
21   LCA(const vector<vector<int>>& _adj, int root)
22   : adj{ _adj }, n{adj.size()}, d(n, log2(n)) {
23       log2[1] = 0;
24       for (int i{2}; i < log2.size(); ++i)
25         log2[i] = log2[i / 2] + 1;
26       an.assign(n, vector<int>(log2[n - 1] + 1, -1));
27       dfs(root);
28   }
29   int operator()(int u, int v) {
30       if (d[u] > d[v]) swap(u, v);
31
32       for (int i{log2[d[v] - d[u]]}; i >= 0; --i)
33         if (d[v] - d[u] >= (1 << i)) v = an[v][i];
34       // v 先走到跟 u 同高度
35       if (u == v) return u;
36
37       for (int i{log2[d[u]]}; i >= 0; --i)
38         if (an[u][i] != an[v][i]) u = an[u][i], v = an[v][i];
39       // u, v 一起走到 lca(u, v) 的下方
40       return an[u][0];
41       // 回傳 lca(u, v)
42   }
43 };
44
45 int main() {
46   int n, q;
47   cin >> n >> q;
48   vector<vector<int>> adj(n);
49
50   for (int i = 1; i < n; ++i) {
51       int u;
52       cin >> u;
53       u--;
54       adj[u].pb(i);
55       adj[i].pb(u);
56   }
57
58   // adj, root
59   LCA lca(adj, 0);
60
61   while (q--) {
62       int u, v;
63       cin >> u >> v;
64       u--, v--;
65       cout << lca(u, v) + 1 << "\\n";
66   }

```

6.15 maximum cardinality matching

```

1 // maximum matching
2 // Edmonds' Blossom algorithm
3 // O(VEa(E,V))
4 class graph {
5     int n;
6     vector<int> p, d, a, c1, c2;
7     // (alternating tree), (unvisited: -1, even: 0, odd: 1), (auxiliary for lca), (cross edge)
8     int label{0};
9     /* DSU */
10    vector<int> dsu_p, dsu_sz, dsu_b;
11    void dsu_reset() {
12        fill(dsu_p.begin(), dsu_p.end(), -1);
13        fill(dsu_sz.begin(), dsu_sz.end(), 1);
14        iota(dsu_b.begin(), dsu_b.end(), 0);
15    }
16    int find(int x) { // collapsing find
17        return dsu_p[x] == -1 ? x : dsu_p[x] = find(dsu_p[x]);
18    }
19    int base(int x) { return dsu_b[find(x)]; }
20    void contract(int x, int y) { // weighted union
21        auto rx{find(x)}, ry{find(y)};
22        if (rx == ry) return;
23        auto b{dsu_b[rx]}; // treat x's base as new base
24        if (dsu_sz[rx] < dsu_sz[ry]) swap(rx, ry);
25        dsu_p[ry] = rx, dsu_sz[rx] += dsu_sz[ry], dsu_b[rx] = b;
26    }
27    /******
28    queue<int> qu{}; // only even vertices
29    void handle_one_side(int x, int y, int b) {
30        for (int v{base(x)}; v != b; v = base(p[v])) {
31            c1[v] = x, c2[v] = y, contract(b, v);
32            if (d[v] == 1) qu.push(v); // odd vertex -> even vertex
33        }
34    }
35    int lca(int u, int v) {
36        ++label; // use next number in order not to clear a
37        while (true) {
38            if (a[u] == label) return u;
39            a[u] = label;
40            if (p[u] != -1) u = base(p[u]);
41            swap(u, v);
42        }
43    }

```

return 0;

```

44 void augment(int r, int y) {
45     if (r == y) return;
46     if (d[y] == 0) { // even vertex -> odd vertex
47         // check d[y] == 0 instead of d[p[y]] == 1, so (m[y], y) is in the blossom
48         augment(m[y], m[c1[y]]);
49         augment(r, m[c2[y]]);
50         m[c1[y]] = c2[y], m[c2[y]] = c1[y];
51     } else {
52         int x{p[y]};
53         augment(r, m[x]);
54         m[x] = y, m[y] = x;
55     }
56 }
57 bool bfs(int r) {
58     dsu_reset();
59     fill(d.begin(), d.end(), -1);
60
61     qu = {}, qu.push(r), d[r] = 0;
62     while (!qu.empty()) {
63         int x{qu.front()}; qu.pop();
64         for (auto& y : adj[x]) {
65             if (base(x) == base(y)) continue;
66             if (d[y] == -1) {
67                 p[y] = x, d[y] = 1;
68                 if (m[y] == -1) { // augmenting path
69                     augment(-1, y);
70                     return true;
71                 } else {
72                     p[m[y]] = y, d[m[y]] = 0;
73                     qu.push(m[y]);
74                 }
75             } else if (d[base(y)] == 0) { // blossom
76                 int b{lca(base(x), base(y))};
77                 handle_one_side(x, y, b);
78                 handle_one_side(y, x, b);
79             }
80         }
81     }
82     return false;
83 }
84 public:
85     vector<vector<int>> adj;
86     vector<int> m;
87     explicit graph(int _n) : n{ _n }, p(n, -1), d(n), a(n), c1(n), c2(n), dsu_p(n), dsu_sz(n), dsu_b(n), adj(n), m(n, -1) {}
88
89     int Edmonds() {
90         int c{0};
91         for (int v{0}; v < n; ++v)
92             if (m[v] == -1 && bfs(v)) ++c;
93         return c;
94     }
95 };

```

```

96 // usage example:
97 int main() {
98     int n = 5;
99     graph g(n);
100     auto add = [&](int u, int v) {
101         g.adj[u].push_back(v);
102         g.adj[v].push_back(u);
103     };
104
105     // sample graph
106     add(0, 1);
107     add(0, 2);
108     add(1, 2); // triangle (0,1,2)
109     add(2, 3);
110     add(3, 4);
111
112     int match_size = g.Edmonds();
113     cout << "Maximum matching size: " << match_size << "\\n";
114     for (int i = 0; i < n; ++i)
115         if (g.m[i] != -1 && i < g.m[i])
116             cout << i << " - " << g.m[i] << "\\n";
117 }

```

6.16 topological sort

```

1 // topological sort 1
2 optional<vector<int>> top_sort(vector<vector<int>>& adj) {
3     vector<int> res{};
4     int n{static_cast<int>(adj.size())};
5     vector<int> cnt(n, 0); // predecessor count
6     for (int u = 0; u < n; ++u)
7         for (auto& v : adj[u]) ++cnt[v];
8
9     queue<int> qu{};
10    for (int u = 0; u < n; ++u) if (!cnt[u]) qu.push(u);
11    while (!qu.empty()) {
12        auto u = qu.front();
13        qu.pop();
14        res.push_back(u);
15
16        for (auto& v : adj[u])
17            if (!--cnt[v]) qu.push(v);
18    }
19
20    if (res.size() != adj.size()) return nullopt;
21    return res;
22 }

```

6.17 tree diameter

```

1 int diam = 0;
2
3 int dfs(int u, int p = -1) {
4     int mx = 0;

```



```

5   for (int v : adj[u]) {
6       if (v != p) {
7           int len = 1 + dfs(v, u);
8           diam = max(diam, mx + len);
9           mx = max(mx, len);
10      }
11  }
12  return mx;
13 }

```

6.18 all longest path

```

1  int fir[maxn]; // Length of the longest
2  // downward path from u into its subtree.
3  int sec[maxn]; // Length of the second
4  // longest downward path from u
5  int res[maxn];
6
7  void dfs1(int u, int p) {
8      for (int v : adj[u]) {
9          if (v != p) {
10             dfs1(v, u);
11             if (fir[v] + 1 > fir[u]) {
12                 sec[u] = fir[u];
13                 fir[u] = fir[v] + 1;
14             }
15             else if (fir[v] + 1 > sec[u]) {
16                 sec[u] = fir[v] + 1;
17             }
18         }
19     }
20     // to_p: the best path length that comes
21     // from the parent's side
22     void dfs2(int u, int p, int to_p) {
23         res[u] = max(to_p, fir[u]);
24
25         for (int v : adj[u]) {
26             if (v != p) {
27                 if (fir[v] + 1 == fir[u]) {
28                     dfs2(v, u, max(to_p, sec[u])
29                         + 1);
30                 }
31                 else {
32                     dfs2(v, u, res[u] + 1);
33                 }
34             }
35         }
36     }
37     // usage
38     dfs1(1, 0);
39     dfs2(1, 0, 0);
40     // Now res[i] is the maximum distance from
41     // node i to any other node
42     for (int i = 1; i <= n; i++) {
43         cout << res[i] << " ";
44     }
45 }

```

6.19 stable matching

```

1  // Stable Marriage Problem (stable matching)
2  // Gale-Shapley algorithm
3  class bipartiteGraph {
4  public:
5      int n;
6      vector<vector<int>> px, py;
7      vector<int> mx, my;
8
9      bipartiteGraph(int _n) : n(_n), px(n,
10         vector<int>(n)), py(n, vector<int>(n
11         )), mx(n, -1), my(n, -1) {}
12      void Gale_Shapley() {
13         queue<int> qu{};
14         vector<priority_queue<pair<int, int
15             >>> pq(n);
16         for (int x{0}; x < n; ++x) {
17             qu.push(x);
18             for (int y{0}; y < n; ++y) pq[x
19                 ].emplace(px[x][y], y);
20         }
21         while (!qu.empty()) {
22             auto x{qu.front()}; qu.pop();
23
24             int y;
25             do {
26                 y = pq[x].top().second; pq[x
27                     ].pop();
28             } while (my[y] != -1 && py[y][x]
29                 <= py[y][my[y]]);
30             if (my[y] != -1) mx[my[y]] = -1,
31                 qu.push(my[y]); // y
32                 // prefers x to my[y]
33             mx[x] = y, my[y] = x;
34         }
35     }
36 };
37
38 int main() {
39     int n = 3;
40     bipartiteGraph g(n);
41
42     // Men preferences (from most to Least
43     // preferred)
44     vector<vector<int>> men = {
45         {0, 1, 2},
46         {1, 2, 0},
47         {1, 0, 2}
48     };
49
50     // Women preferences (from most to Least
51     // preferred)
52     vector<vector<int>> women = {
53         {1, 0, 2},
54         {2, 1, 0},
55         {0, 1, 2}
56     };
57
58     // Convert orders to scores (higher is
59     // better)
60     for (int x = 0; x < n; ++x)
61         for (int pos = 0; pos < n; ++pos)

```

```

53         g.px[x][men[x][pos]] = n - pos;
54
55     for (int y = 0; y < n; ++y)
56         for (int pos = 0; pos < n; ++pos)
57             g.py[y][women[y][pos]] = n - pos;
58
59     g.Gale_Shapley();
60
61     // Output matches: man x is matched to
62     // woman g.mx[x]
63     for (int x = 0; x < n; ++x)
64         cout << x << " " << g.mx[x] << '\n';
65
66     return 0;
67 }

```

6.20 tree diameter (len,end)

```

1  array<int, 2> dfs(int u, int w = -1) {
2      array<int, 2> mx{0, u}; // {length,
3      // farthest Leaf}
4      for (auto& v : adj[u]) {
5          if (v == w) continue;
6          auto [len, leaf]{dfs(v, u)};
7          mx = max(mx, {len + 1, leaf});
8      }
9      return mx;
10 }
11
12 array<int, 3> tree_diameter(int a = 0) {
13     auto b{dfs(a)[1]}; // farthest node
14     // from 'a'
15     auto [l, c]{dfs(b)}; // farthest node
16     // from 'b'
17     return {l, b, c}; // {diameter
18     // length, endpoint1, endpoint2}
19 }

```

6.21 Bellman-Ford

```

1  // Bellman-Ford algorithm
2  template<typename T>
3  optional<vector<optional<T>>> Bellman_Ford(
4      const vector<vector<pair<int, T>>>& adj,
5      int s) {
6      const auto& n{adj.size()};
7      vector<optional<T>> d(n, nullopt);
8      d[s] = 0;
9
10     queue<int> qu{};
11     vector<bool> in(n, false), in2(n, false);
12
13     qu.push(s); in[s] = true;
14     for (int i{0}; i < n; ++i) { // at most
15         // n-1 edges
16         while (!qu.empty()) {
17             int u{qu.front()};
18             qu.pop(); in[u] = false;
19             for (auto& [v, w] : adj[u])

```

```

16         if (!d[v] || d[v] > d[u].
17             value() + w) { // relax
18             d[v] = d[u].value() + w;
19             if (!in2[v]) qu2.push(v);
20             in2[v] = true;
21         }
22     }
23     qu.swap(qu2), in.swap(in2);
24 }
25
26 if (qu.empty()) return d;
27 return nullopt; // if negative cycle

```

7 Language

7.1 CNF

```

1  #define MAXN 55
2  struct CNF{
3      int s,x,y;//s->xy | s->x, if y==-1
4      int cost;
5      CNF(){}
6      CNF(int s,int x,int y,int c):s(s),x(x),y(y
7          ),cost(c){}
8  };
9  int state;//規則數量
10 map<char,int> rule;//每個字元對應到的規則
11 //小寫字母為終端字符
12 vector<CNF> cnf;
13 void init(){
14     state=0;
15     rule.clear();
16     cnf.clear();
17 }
18 void add_to_cnf(char s,const string &p,int
19     cost){
20     //加入一個s -> <p>的文法，代價為cost
21     if(rule.find(s)==rule.end())rule[s]=state
22         ++;
23     for(auto c:p)if(rule.find(c)==rule.end())
24         rule[c]=state++;
25     if(p.size()==1){
26         cnf.push_back(CNF(rule[s],rule[p[0]],-1,
27             cost));
28     }else{
29         int left=rule[s];
30         int sz=p.size();
31         for(int i=0;i<sz-2;++i){
32             cnf.push_back(CNF(left,rule[p[i]],
33                 state,0));
34             left=state++;
35         }
36         cnf.push_back(CNF(left,rule[p[sz-2]],
37             rule[p[sz-1]],cost));
38     }
39 }
40 vector<long long> dp[MAXN][MAXN];
41 vector<bool> neg_INF[MAXN][MAXN]; //如果花費
42 //是負的可能會有無限小的情形

```

```

34 void relax(int l, int r, const CNF &c, long
    long cost, bool neg_c=0){
35     if(!neg_INF[l][r][c.s]&&(neg_INF[l][r][c.x
        ]||cost<dp[l][r][c.s])){
36         if(neg_c||neg_INF[l][r][c.x]){
37             dp[l][r][c.s]=0;
38             neg_INF[l][r][c.s]=true;
39         }else dp[l][r][c.s]=cost;
40     }
41 }
42 void bellman(int l, int r, int n){
43     for(int k=1; k<=state;++k)
44         for(auto c:cnf)
45             if(c.y==-1)relax(l,r,c,dp[l][r][c.x]+c
                .cost,k=n);
46 }
47 void cyk(const vector<int> &tok){
48     for(int i=0; i<(int)tok.size();++i){
49         for(int j=0; j<(int)tok.size();++j){
50             dp[i][j]=vector<long long>(state+1,
                INT_MAX);
51             neg_INF[i][j]=vector<bool>(state+1,
                false);
52         }
53         dp[i][i][tok[i]]=0;
54         bellman(i,i,tok.size());
55     }
56     for(int r=1; r<(int)tok.size();++r){
57         for(int l=r-1; l>=0; --l){
58             for(int k=l; k<r; ++k)
59                 for(auto c:cnf)
60                     if(~c.y)relax(l,r,c,dp[l][k][c.x]+
                        dp[k+1][r][c.y]+c.cost);
61             bellman(l,r,tok.size());
62         }
63     }
64 }

```

8 Number Theory

8.1 Linear Sieve

```

1 // Calculate the smallest divisor of
  integers in [2, maxn] in O(maxn)
2 vector<int> min_div[] {
3     constexpr int maxn = 400000 + 10;
4
5     vector<int> v(maxn), p;
6
7     for (int i = 2; i < maxn; ++i) {
8         if (!v[i]) {
9             v[i] = i;
10            p.push_back(i);
11        }
12        for (int j = 0; p[j] * i < maxn; ++j)
13            {
14                v[p[j] * i] = p[j];
15                if (p[j] == v[i]) break;
16            }
17    }

```

```

18     return v;
19 }();

```

8.2 C(n,m)

```

1 ll Cnm(ll n, ll m) {
2     if (m > n / 2) m = n - m;
3     ll r[1];
4     for (ll i{1}, j{n}; i <= m; ++i, --j) r
        *= j, r /= i;
5     return r;
6 }

```

8.3 Euler Totient 1 to n

```

1 void phi_1_to_n(int n) {
2     vector<int> phi(n + 1);
3     for (int i = 0; i <= n; i++)
4         phi[i] = i;
5
6     for (int i = 2; i <= n; i++) {
7         if (phi[i] == i) {
8             for (int j = i; j <= n; j += i)
9                 phi[j] -= phi[j] / i;
10        }
11    }
12 }

```

8.4 derangement (Principle of Inclusion-Exclusion)

```

1 // 1. Principle of Inclusion-Exclusion
2 // n! = n! * Σ (from k=0 to n) [((-1)^k) / (
    k!)]
3
4 mint c = 1;
5 for (int i = 1; i <= n; i++) {
6     c = (c * i) + (i % 2 == 1 ? -1 : 1);
7     cout << c.val() << ' ';
8 }

```

8.5 matrix template (with fast power)

```

1 template<class T> struct Matrix {
2     T **mat; int a, b;
3
4     Matrix() : a(0), b(0) {}
5     Matrix(int a_, int b_) : a(a_), b(b_) {
6         int i, j;
7         mat = new T*[a];
8         for (i = 0; i < a; ++i) {

```

```

9             mat[i] = new T[b];
10            for (j = 0; j < b; ++j) {
11                mat[i][j] = 0;
12            }
13        }
14    }
15    Matrix(const vector<vector<T>>& vt) {
16        int i, j;
17
18        *this = Matrix((int)vt.size(), (int)
            vt[0].size());
19        for (i = 0; i < a; ++i) {
20            for (j = 0; j < b; ++j) {
21                mat[i][j] = vt[i][j];
22            }
23        }
24    }
25    Matrix operator*(const Matrix& m) {
26        int i, j, k;
27
28        assert(b == m.a);
29        Matrix r(a, m.b);
30        for (i = 0; i < a; ++i) {
31            for (j = 0; j < m.b; ++j) {
32                for (k = 0; k < b; ++k) {
33                    r.mat[i][j] += mul(mat[i
                        ][k], m.mat[k][j]);
34                    r.mat[i][j] %= MOD;
35                }
36            }
37        }
38        return r;
39    }
40    Matrix& operator*=(const Matrix& m) {
41        return *this = (*this) * m;
42    }
43    friend Matrix power(Matrix m, long long
        p) {
44        int i;
45
46        assert(m.a == m.b);
47        Matrix r(m.a, m.b);
48        for (i = 0; i < m.a; ++i) {
49            r.mat[i][i] = 1;
50        }
51        for (; p > 0; p >>= 1, m *= m) {
52            if (p & 1) {
53                r *= m;
54            }
55        }
56        return r;
57    }
58 };
59
60 int main() {
61     Matrix<int> mat(adj);
62     mat = power(mat, k); // mat^k
63     cout << mat.mat[i][j];
64 }

```

8.6 Sieve of Eratosthenes (with big num)

```

1 const int MX = 100000;
2 bool np[MX + 1];
3 vector<int> prime_numbers;
4
5 int init = []() {
6     np[0] = np[1] = true;
7     for (int i = 2; i <= MX; i++) {
8         if (!np[i]) {
9             prime_numbers.push_back(i);
10            for (int j = i; j <= MX / i; j
                ++){ // 避免溢出的写法
11                np[i * j] = true;
12            }
13        }
14    }
15    return 0;
16 }();
17
18 bool is_prime(long long n) {
19     if (n <= MX) {
20         return !np[n];
21     }
22     for (long long p : prime_numbers) {
23         if (p * p > n) {
24             break;
25         }
26         if (n % p == 0) {
27             return false;
28         }
29     }
30     return true;
31 }

```

8.7 mod inv

```

1 // Modular inverse when mod is prime
2 long long mod_inverse(long long a, long long
    mod) {
3     return power_mod(a, mod - 2, mod); //
    Fermat's Little Theorem
4 }

```

8.8 derangement (DP)

```

1 // 2. DP
2 // !n = (n - 1) * (!n - 1 + !n - 2)),
    with !0 = 1, !1 = 0
3
4 mint a = 1, b = 0;
5 cout << 0 << ' ';
6
7 for (int i = 2; i <= n; i++) {
8     mint c = (i - 1) * (a + b);
9     cout << c.val() << ' ';
10    a = b;
11    b = c;
12 }

```

8.9 Euler Totient

```

1 int phi(int n) {
2     int result = n;
3     for (int i = 2; i * i <= n; i++) {
4         if (n % i == 0) {
5             while (n % i == 0)
6                 n /= i;
7             result -= result / i;
8         }
9     }
10    if (n > 1)
11        result -= result / n;
12    return result;
13 }

```

8.10 fast power

```

1 /* Iterative Function to calculate (a^b) %
   mod in O(Log b) */
2 long long power_mod(long long a, long long b
3 , long long mod) {
4     long long res=1;
5     while (b > 0) {
6         if (b & 1) res = res * a % mod;
7         b >>= 1;
8         a = (a * a) % mod;
9     }
10    return res;
11 }

```

8.11 first and second mex

```

1 // Calculate first and second MEX
2 pair<int, int> calculate_mexes(vector<int>&
3 nums) {
4     int n = nums.size();
5     vector<bool> seen(n + 2, false);
6
7     for (int num : nums) {
8         if (num >= 0 && num < seen.size()) {
9             seen[num] = true;
10        }
11    }
12
13    int first_mex = -1;
14    int second_mex = -1;
15
16    for (int i = 0; i < seen.size(); ++i) {
17        if (!seen[i]) {
18            if (first_mex == -1) {
19                first_mex = i;
20            } else {
21                second_mex = i;
22                break;
23            }
24        }
25    }
26 }

```

```

26     return {first_mex, second_mex};
27 }

```

8.12 Catalan Number

```

1 // Catalan Number
2 // C(n) = 1/(n+1) * (2n choose n) = (2n)! /
3 // ((n+1)! * n!)
4 ll catalan(int n) {
5     return fact[2 * n] * invf[n + 1] % mod *
6     invf[n] % mod;
7 }

```

8.13 Chinese Remainder Theorem

```

1 // coprime (p^k)
2 pair<ll, ll> CRT(const vector<pair<ll, ll>&
3 congruences) {
4     ll M = 1, sol = 0;
5     for (auto& [m, a] : congruences) M *= m;
6     for (auto& [m, a] : congruences) {
7         ll x = M / m, y = MI(x, m);
8         sol = MA(sol, MM(MM(a, x, M), y, M),
9 M);
10    }
11    return {M, sol};
12 }
13 // example usage
14 int main() {
15     // x ≡ 2 (mod 3)
16     // x ≡ 3 (mod 5)
17     // x ≡ 2 (mod 7)
18     vector<pair<ll, ll>> congruences = {{3,
19 2}, {5, 3}, {7, 2}};
20     auto [M, sol] = CRT(congruences);
21     cout << "M: " << M << ", x: " << sol <<
22 endl; // M: 105, x: 23
23 }

```

8.14 mod inv (not prime)

```

1 /* exists when a and mod are coprime */
2 /* but mod is not prime */
3 long long MI(long long a, long long mod) {
4     return power_mod(a, euler_phi(mod) - 1,
5 mod);
6 }

```

8.15 mod inv (not coprime)

```

1 /* a and mod are not coprime */
2 long long MI(long long a, long long mod) {
3     long long d, x, y;
4     extEcu(a, mod, d, x, y);
5     return d == 1 ? (x + mod) % mod : -1;
6 }

```

8.16 Mint

```

1 using ll = long long;
2 const ll PM{1000000007};
3 ll MC(ll a) { return (a % PM + PM) % PM; }
4 // for negative a
5 ll MA(ll a, ll b) { return (a + b) % PM; }
6 // (a + b) % PM
7 ll MS(ll a, ll b) { return (a - b + PM) % PM; }
8 // (a - b) % PM
9 ll MM(ll a, ll b) { return (a * b) % PM; }
10 // (a * b) % PM
11 ll MP(ll a, ll b) { return (a ^ b) % PM; }
12 ll res[1];
13 do {
14     if (b & 1) res = MM(res, a);
15     while (a = MM(a, a), b >>= 1);
16     return res;
17 }
18 ll MI(ll a) { return MP(a, PM - 2); } //
19 modular inverse of a % PM
20 ll MD(ll a, ll b) { return MM(a, MI(b)); }
21 // (a / b) % PM

```

8.17 C(n,k) mod inverse

```

1 fac[0] = 1;
2 for (int i = 1; i <= n; ++i) {
3     fac[i] = fac[i - 1] * i % MOD;
4 }
5 inv_fac[n] = power_mod(fac[n], MOD - 2, MOD);
6 ;
7 for (int i = n - 1; i >= 0; --i) {
8     inv_fac[i] = inv_fac[i + 1] * (i + 1) %
9 MOD;
10 }
11 // C(n, k) = fac[n] * inv_fac[k] * inv_fac[n
12 - k];

```

8.18 Linear Diophantine 丢番圖

```

1 // ax + by = c
2 // x0 = x * (c / gcd(a, b))
3 // y0 = y * (c / gcd(a, b))
4 // x = x0 + k * (b / gcd(a, b))
5 // y = y0 - k * (a / gcd(a, b))
6 // k is any integer

```

```

7 tll extendedEuclid(ll a, ll b) {
8     if (b == 0) return {a, 1, 0};
9     ll d, x1, y1;
10    tie(d, x1, y1) = extendedEuclid(b, a % b);
11    return {d, y1, x1 - (a / b) * y1};
12 }
13 }
14
15 pll linearDiophantine(ll a, ll b, ll c) {
16     ll d, x, y;
17     tie(d, x, y) = extendedEuclid(a, b);
18     if (c % d != 0) return {-1, -1}; // no
19     solution
20     x *= c / d;
21     y *= c / d;
22     return {x, y};
23 }
24 // Example usage
25 int main() {
26     ll a = 15, b = 10, c = 35;
27     pll res = linearDiophantine(a, b, c);
28     if (res.first == -1 && res.second == -1)
29     {
30         cout << "No solution exists" << endl;
31     }
32     else {
33         cout << "One solution is x = " <<
34         res.first << ", y = " <<
35         res.second << endl;
36     }
37     return 0;
38 }
39 ax + by = c

```

8.19 擴展歐基里德

```

1 /* solve x, y for ax + by = gcd(a, b) = g */
2 template<typename T>
3 void extEcu(T a, T b, T &g, T &x, T &y) {
4     if (b) extEcu(b, a % b, g, y, x), y -= x
5     * (a / b);
6     else g = a, x = 1, y = 0;
7 }

```

8.20 Sieve of Eratosthenes

```

1 void sieve(vector<int>& primes) {
2     vector<int> is_prime(INF + 1, 0);
3     is_prime[0] = 1;
4     is_prime[1] = 1;
5
6     int sq = sqrt(INF);
7
8     for (int i = 2; i <= sq; ++i) {
9         if (!is_prime[i]) {
10            primes.push_back(i);
11            for (int j = i * i; j <= INF; j
12 += i) {

```

```

12         is_prime[j] = 1;
13     }
14 }
15 }
16 }

```

8.21 builtin

```

1 // __builtin_ctz
2 // 這個函數作用是返回輸入數二進制表示從最低
  // 位開始 (右起) 的連續的0的個數; 如果傳入0
  // 則行為未定義。
3
4 int __builtin_ctz (unsigned int x)
5 Returns the number of trailing 0-bits in x,
  // starting at the least significant bit
  // position. If x is 0, the result is
  // undefined.
6
7 int __builtin_ctzl (unsigned long)
8 Similar to __builtin_ctz, except the
  // argument type is unsigned long.
9
10 int __builtin_ctzll (unsigned long long)
11 Similar to __builtin_ctz, except the
  // argument type is unsigned long long.
12
13
14 // __builtin_clz
15 // 這個函數作用是返回輸入數二進制表示從最高
  // 位開始 (左起) 的連續的0的個數; 如果傳入0
  // 則行為未定義。
16
17 int __builtin_clz (unsigned int x)
18 Returns the number of leading 0-bits in x,
  // starting at the most significant bit
  // position. If x is 0, the result is
  // undefined.
19
20 int __builtin_clzl (unsigned long)
21 Similar to __builtin_clz, except the
  // argument type is unsigned long.
22
23 int __builtin_clzll (unsigned long long)
24 Similar to __builtin_clz, except the
  // argument type is unsigned long long.
25
26
27 // __builtin_ffs
28 // 這個函數作用是返回輸入數二進制表示的最低
  // 非0位位的下標。下標從1開始計數; 如果傳入0
  // 則返回0
29
30 int __builtin_ffs (unsigned int x)
31 Returns one plus the index of the least
  // significant 1-bit of x, or if x is zero,
  // returns zero.
32
33 int __builtin_ffsl (unsigned long)
34 Similar to __builtin_ffs, except the
  // argument type is unsigned long.

```

```

35 int __builtin_ffsll (unsigned long long)
36 Similar to __builtin_ffs, except the
  // argument type is unsigned long long.
37
38
39 // __builtin_popcount
40 // 這個函數作用是返回輸入數二進制表示中1的個
  // 數。
41
42 int __builtin_popcount (unsigned int x)
43 Returns the number of 1-bits in x.
44
45 int __builtin_popcountl (unsigned long)
46 Similar to __builtin_popcount, except the
  // argument type is unsigned long.
47
48 int __builtin_popcountll (unsigned long long)
49 Similar to __builtin_popcount, except the
  // argument type is unsigned long long.
50
51
52 // __builtin_parity
53 // 這個函數作用是返回輸入數二進制表示中1的個
  // 數的奇偶性。1表示奇數個1。0表示偶數個1。
54
55 int __builtin_parity (unsigned int x)
56 Returns the parity of x, i.e. the number of
  // 1-bits in x modulo 2.
57
58 int __builtin_parityl (unsigned long)
59 Similar to __builtin_parity, except the
  // argument type is unsigned long.
60
61 int __builtin_parityll (unsigned long long)
62 Similar to __builtin_parity, except the
  // argument type is unsigned long long.

```

8.22 C(n,k) DP

```

1 long long binomial(long long n, long long k,
  // long long p) {
2     // dp[i][j] = iCj
3     vector<vector<long long>> dp(n + 1,
  // vector<long long>(k + 1, 0));
4
5     for (int i = 0; i <= n; ++i) {
6         dp[i][0] = 1;
7         if (i <= k) dp[i][i] = 1;
8     }
9
10    for (int i = 0; i <= n; ++i) {
11        for (int j = 1; j <= min(i, k); ++j)
12            if (i != j) {
13                dp[i][j] = (dp[i - 1][j - 1]
14                    + dp[i - 1][j]) % p;
15            }
16    }
17 }

```

```

18     return dp[n][k];
19 }

```

9 Range Queries

9.1 subarray maximum sum queries

```

1 // CSES: Subarray Sum Queries
2
3 const int maxn = 2e5;
4 int n = 0;
5 int t[maxn << 1];
6
7 void build() {
8     for (int i = 0; i < n; ++i) {
9         t[i + n] = 0;
10    }
11    for (int i = n - 1; i >= 1; --i) {
12        t[i] = t[i << 1] + t[i << 1 | 1];
13    }
14 }
15
16 void update(int idx, int val) {
17     idx += n;
18     if (t[idx] == val) return;
19     t[idx] = val;
20     for (idx >= 1; idx >= 1; idx >= 1) {
21         t[idx] = t[idx << 1] + t[idx << 1 | 1];
22     }
23 }
24
25 int query(int l, int r) {
26     int ans = 0;
27     for (l += n, r += n; l < r; l >= 1, r
28         >= 1) {
29         if (l & 1) ans += t[l++];
30         if (r & 1) ans += t[--r];
31     }
32     return ans;
33 }
34
35 void solve() {
36     int q;
37     cin >> q;
38     n = 0;
39     vector<pii> op(q);
40     map<int, int> mp;
41     vector<int> to_val;
42
43     memset(t, 0, sizeof(t));
44
45     for (int i = 0; i < q; ++i) {
46         char c;
47         cin >> c >> op[i].second;
48         if (c == 'I') {
49             op[i].first = 0;
50             mp[op[i].second] = 1;
51         }
52         else if (c == 'D') {
53             op[i].first = 1;

```

```

53         mp[op[i].second] = 1;
54     }
55     else if (c == 'K') {
56         op[i].first = 2;
57     }
58     else {
59         op[i].first = 3;
60     }
61 }
62
63 for (auto& x : mp) {
64     x.second = n;
65     to_val.push_back(x.first);
66     n++;
67 }
68
69 build();
70
71 for (pii o : op) {
72     if (o.first == 0) {
73         int idx = mp[o.second];
74         update(idx, 1);
75     }
76     if (o.first == 1) {
77         int idx = mp[o.second];
78         update(idx, 0);
79     }
80     if (o.first == 2) {
81         int k = o.second;
82         int ans = -1;
83         int l = 0, r = n - 1;
84         while (l <= r) {
85             int m = l + (r - l) / 2;
86             int rk = query(0, m + 1);
87             if (rk >= k) {
88                 ans = m;
89                 r = m - 1;
90             }
91             else {
92                 l = m + 1;
93             }
94         }
95         if (ans == -1) cout << "invalid\
96             n";
97         else cout << to_val[ans] << "\n";
98     }
99     if (o.first == 3) {
100         auto it = mp.lower_bound(o.
101             second);
102         if (it == mp.begin() && (it ==
103             mp.end() || o.second <= it->
104             first)) {
105             cout << "0\n";
106         }
107         else if (it == mp.end()) {
108             cout << query(0, n) << "\n";
109         }
110         else {
111             cout << query(0, it->second)
112                 << "\n";
113         }
114     }
115 }

```

9.2 find first equal or greater

```

1 // CSE: Hotel Queries
2
3 int n, m;
4 int sz = 1;
5 const int maxn = 2e6;
6 int t[maxn << 1];
7
8 void build(const vector<int>& a) {
9     for (int i = 0; i < n; ++i) {
10         t[i + sz] = a[i];
11     }
12     for (int i = sz - 1; i >= 1; --i) {
13         t[i] = max(t[i << 1], t[i << 1 | 1]);
14     }
15 }
16
17 void update(int i, int val) {
18     i += sz;
19     t[i] = val;
20     for (i >= 1; i >= 1; i >= 1) {
21         t[i] = max(t[i << 1], t[i << 1 | 1]);
22     }
23 }
24
25 int query(int r) {
26     if (t[1] < r) return 0;
27
28     int i = 1;
29     while (i < sz) {
30         if (t[i << 1] >= r) i = i << 1;
31         else i = i << 1 | 1;
32     }
33     return i - sz + 1;
34 }
35
36 void solve() {
37     cin >> n >> m;
38     vector<int> a(n);
39
40     for (int i = 0; i < n; ++i) {
41         cin >> a[i];
42     }
43
44     while (sz < n) sz <= 1;
45     memset(t, 0, sizeof(t));
46     build(a);
47
48     int r;
49     while (m--) {
50         cin >> r;
51         int idx = query(r);
52         cout << idx << " ";
53         if (idx) {
54             a[idx - 1] -= r;
55             update(idx - 1, a[idx - 1]);
56         }
57     }
58     cout << "\n";
59 }

```

9.3 find k-th and count less than

```

1 /*
2 SPOJ: ORDERSET - Order statistic set
3
4 In this problem, you have to maintain a
5 dynamic set of numbers which support the
6 two fundamental operations
7
8 INSERT(S,x): if x is not in S, insert x into S
9 DELETE(S,x): if x is in S, delete x from S
10 and the two type of queries
11
12 K-TH(S) : return the k-th smallest element of S
13 COUNT(S,x): return the number of elements of S smaller than x
14 */
15
16 const int maxn = 2e5;
17 int n = 0;
18 int t[maxn << 1];
19
20 void build() {
21     for (int i = 0; i < n; ++i) {
22         t[i + n] = 0;
23     }
24     for (int i = n - 1; i >= 1; --i) {
25         t[i] = t[i << 1] + t[i << 1 | 1];
26     }
27 }
28
29 void update(int idx, int val) {
30     idx += n;
31     if (t[idx] == val) return;
32     t[idx] = val;
33     for (idx >= 1; idx >= 1; idx >= 1) {
34         t[idx] = t[idx << 1] + t[idx << 1 | 1];
35     }
36 }
37
38 int query(int l, int r) {
39     int ans = 0;
40     for (l += n, r += n; l < r; l >= 1, r >= 1) {
41         if (l & 1) ans += t[l++];
42         if (r & 1) ans += t[--r];
43     }
44     return ans;
45 }
46
47 void solve() {
48     int q;
49     cin >> q;
50     n = 0;
51     vector<pii> op(q);
52     map<int, int> mp;
53     vector<int> to_val;
54
55     memset(t, 0, sizeof(t));
56
57     for (int i = 0; i < q; ++i) {
58         char c;

```

```

59         cin >> c >> op[i].second;
60         if (c == 'I') {
61             op[i].first = 0;
62             mp[op[i].second] = 1;
63         }
64         else if (c == 'D') {
65             op[i].first = 1;
66             mp[op[i].second] = 1;
67         }
68         else if (c == 'K') {
69             op[i].first = 2;
70         }
71         else {
72             op[i].first = 3;
73         }
74     }
75
76     for (auto& x : mp) {
77         x.second = n;
78         to_val.push_back(x.first);
79         n++;
80     }
81
82     build();
83
84     for (pii o : op) {
85         if (o.first == 0) {
86             int idx = mp[o.second];
87             update(idx, 1);
88         }
89         if (o.first == 1) {
90             int idx = mp[o.second];
91             update(idx, 0);
92         }
93         if (o.first == 2) {
94             int k = o.second;
95             int ans = -1;
96             int l = 0, r = n - 1;
97             while (l <= r) {
98                 int m = l + (r - l) / 2;
99                 int rk = query(0, m + 1);
100                 //printf("(m, rk): %Lld, %Lld\n", m, rk);
101                 if (rk >= k) {
102                     ans = m;
103                     r = m - 1;
104                 }
105                 else {
106                     l = m + 1;
107                 }
108             }
109             if (ans == -1) cout << "invalid\n";
110             else cout << to_val[ans] << "\n";
111         }
112         if (o.first == 3) {
113             auto it = mp.lower_bound(o.second);
114             if (it == mp.begin() && (it == mp.end() || o.second <= it->first)) {
115                 cout << "0\n";
116             }
117             else if (it == mp.end()) {
118                 cout << query(0, n) << "\n";
119             }
120         }
121     }

```

```

117     }
118     else {
119         cout << query(0, it->second) << "\n";
120     }
121 }
122 }
123 }
124 }

```

10 String

10.1 Z

```

1 // 计算并返回 z 数组 · 其中 z[i] = LCP(s[i:], s)
2 vector<int> calc_z(const string& s) {
3     int n = s.size();
4     vector<int> z(n);
5     int box_l = 0, box_r = 0;
6     for (int i = 1; i < n; i++) {
7         if (i <= box_r) {
8             z[i] = min(z[i - box_l], box_r - i + 1);
9         }
10        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
11            box_l = i;
12            box_r = i + z[i];
13            z[i]++;
14        }
15    }
16    z[0] = n;
17    return z;
18 }

```

10.2 manacher

```

1 class Solution {
2 public:
3     int countSubstrings(string s) {
4         int l1 = s.size(), l2 = l1 * 2 + 1;
5         string ch = "#";
6         for (char c : s) {
7             ch = ch + c + "#";
8         }
9
10        int c = 0, r = 0, cnt = 0;
11        vector<int> p(l2);
12        for (int i = 0; i < l2; i++) {
13            p[i] = (i < r)? min(p[2 * c - i], r - i): 1;
14            while (i + p[i] < l2 && i - p[i] >= 0 && ch[i + p[i]] == ch[i - p[i]]) p[i]++;
15            if (i + p[i] > r) {
16                r = i + p[i];
17                c = i;
18            }
19        }
20    }
21 }

```



```

18     }
19     int l = p[i] - 1;
20     if(1 % 2 == 0) cnt += 1 / 2;
21     else cnt += 1 / 2 + 1;
22 }
23 return cnt;
24 }
25 };

```

10.3 AC 自動機

```

1 template<char L='a',char R='z'>
2 class ac_automaton{
3     struct joe{
4         int next[R-L+1],fail,efl,ed,cnt_dp,vis;
5         joe():ed(0),cnt_dp(0),vis(0){
6             for(int i=0;i<=R-L;++i)next[i]=0;
7         }
8     };
9     public:
10     std::vector<joe> S;
11     std::vector<int> q;
12     int qs,qe,vt;
13     ac_automaton():S(1),qs(0),qe(0),vt(0){
14         void clear(){
15             q.clear();
16             S.resize(1);
17             for(int i=0;i<=R-L;++i)S[0].next[i]=0;
18             S[0].cnt_dp=S[0].vis=qs=qe=vt=0;
19         }
20         void insert(const char *s){
21             int o=0;
22             for(int i=0,id;s[i];++i){
23                 id=s[i]-L;
24                 if(!S[o].next[id]){
25                     S.push_back(joe());
26                     S[o].next[id]=S.size()-1;
27                 }
28                 o=S[o].next[id];
29             }
30             ++S[o].ed;
31         }
32         void build_fail(){
33             S[0].fail=S[0].efl=-1;
34             q.clear();
35             q.push_back(0);
36             ++qe;
37             while(qs!=qe){
38                 int pa=q[qs++],id,t;
39                 for(int i=0;i<=R-L;++i){
40                     t=S[pa].next[i];
41                     if(!t)continue;
42                     id=S[pa].fail;
43                     while(~id&&!S[id].next[i])id=S[id].fail;
44                     S[t].fail=~id?S[id].next[i]:0;
45                     S[t].efl=S[S[t].fail].ed?S[t].fail:S[t].fail].efl;
46                     q.push_back(t);
47                     ++qe;
48                 }
49             }
50         }

```

```

51     /*DP出每個前綴在字串s出現的次數並傳回所有
52     字串被s匹配成功的次數O(N*M)*/
53     int match_0(const char *s){
54         int ans=0,id,p=0,t;
55         for(i=0;s[i];++i){
56             id=s[i]-L;
57             while(!S[p].next[id]&&p) p=S[p].fail;
58             if(!S[p].next[id])continue;
59             p=S[p].next[id];
60             ++S[p].cnt_dp; /*匹配成功則它所有後綴都
61             可以被匹配(DP計算)*/
62         }
63         for(i=qe-1;i>=0;--i){
64             ans+=S[q[i]].cnt_dp*S[q[i]].ed;
65             if(~S[q[i]].fail)S[S[q[i]].fail].cnt_dp+=S[q[i]].cnt_dp;
66         }
67         return ans;
68     }
69     /*多串匹配走efl邊並傳回所有字串被s匹配成功
70     的次數O(N*M^1.5)*/
71     int match_1(const char *s) const{
72         int ans=0,id,p=0,t;
73         for(int i=0;s[i];++i){
74             id=s[i]-L;
75             while(!S[p].next[id]&&p) p=S[p].fail;
76             if(!S[p].next[id])continue;
77             p=S[p].next[id];
78             if(S[p].ed)ans+=S[p].ed;
79             for(t=S[p].efl;~t;t=S[t].efl){
80                 ans+=S[t].ed; /*因為都走efl邊所以保證
81                 匹配成功*/
82             }
83         }
84         return ans;
85     }
86     /*枚舉(s的子字串a)的所有相異字串各恰一次
87     並傳回次數O(N*M^(1/3))*/
88     int match_2(const char *s){
89         int ans=0,id,p=0,t;
90         ++vt;
91         /*把截記vt+=1 只要vt沒溢位 所有S[p].vis==vt就會變成false
92         這種利用vt的方法可以O(1)歸零vis陣列*/
93         for(int i=0;s[i];++i){
94             id=s[i]-L;
95             while(!S[p].next[id]&&p) p=S[p].fail;
96             if(!S[p].next[id])continue;
97             p=S[p].next[id];
98             if(S[p].ed&&S[p].vis!=vt){
99                 S[p].vis=vt;
100                 ans+=S[p].ed;
101             }
102             for(t=S[p].efl;~t&&S[t].vis!=vt;t=S[t].efl){
103                 S[t].vis=vt;
104                 ans+=S[t].ed; /*因為都走efl邊所以保證
105                 匹配成功*/
106             }
107         }
108         return ans;
109     }
110     /*把AC自動機變成真的自動機*/

```

10.4 KMP

```

1 void evolution(){
2     for(qs=1;qs!=qe;){
3         int p=q[qs++];
4         for(int i=0;i<=R-L;++i)
5             if(S[p].next[i]==0)S[p].next[i]=S[S[p].fail].next[i];
6     }
7 }
8
9 // 在文本串 text 中查找模式串 pattern · 返回
10 // 所有成功匹配的位置 (pattern[0] 在 text
11 // 中的下标)
12 vector<int> kmp(const string& text, const
13 string& pattern) {
14     int m = pattern.size();
15     vector<int> pi(m);
16     int cnt = 0;
17     for (int i = 1; i < m; i++) {
18         char b = pattern[i];
19         while (cnt && pattern[cnt] != b) {
20             cnt = pi[cnt - 1];
21         }
22         if (pattern[cnt] == b) {
23             cnt++;
24         }
25         pi[i] = cnt;
26     }
27     vector<int> pos;
28     cnt = 0;
29     for (int i = 0; i < text.size(); i++) {
30         char b = text[i];
31         while (cnt && pattern[cnt] != b) {
32             cnt = pi[cnt - 1];
33         }
34         if (pattern[cnt] == b) {
35             cnt++;
36         }
37         if (cnt == m) {
38             pos.push_back(i - m + 1);
39             cnt = pi[cnt - 1];
40         }
41     }
42     return pos;
43 }

```

10.5 suffix array lcp

```

1 #define radix_sort(x,y){\
2     for(i=0;i<A;++i)c[i]=0;\
3     for(i=0;i<n;++i)c[x[y[i]]]++;\
4     for(i=1;i<A;++i)c[i]+=c[i-1];\
5     for(i=n-1;~i;--i)sa[--c[x[y[i]]]]=y[i];\
6 }
7 #define AC(r,a,b)\
8     r[a]=r[b]||a+k>n||r[a+k]!=r[b+k]

```

```

9 void suffix_array(const char *s,int n,int *
10 sa,int *rank,int *tmp,int *c){
11     int A='z'+1,i,k,id=0;
12     for(i=0;i<n;++i)rank[tmp[i]=i]=s[i];
13     radix_sort(rank,tmp);
14     for(k=1;id<n-1;k<=1){
15         for(id=0,i=n-k;i<n;++i)tmp[id++]=i;
16         for(i=0;i<n;++i)
17             if(sa[i]>=k)tmp[id++]=sa[i]-k;
18         radix_sort(rank,tmp);
19         swap(rank,tmp);
20         for(rank[sa[0]]=id=0,i=1;i<n;++i)
21             rank[sa[i]]=id+=AC(tmp,sa[i-1],sa[i]);
22         A=id+1;
23     }
24     //h:高度數組 sa:後綴數組 rank:排名
25     void suffix_array_lcp(const char *s,int len,
26     int *h,int *sa,int *rank){
27         for(int i=0;i<len;++i)rank[sa[i]]=i;
28         for(int i=0,k=0;i<len;++i){
29             if(rank[i]==0)continue;
30             if(k)--k;
31             while(s[i+k]==s[sa[rank[i]-1]+k])++k;
32             h[rank[i]]=k;
33         }
34         h[0]=0; // h[k]=Lcp(sa[k],sa[k-1]);

```

10.6 hash

```

1 #define MAXN 1000000
2 #define mod 1073676287
3 /*mod 必須要是質數*/
4 typedef long long T;
5 char s[MAXN+5];
6 T h[MAXN+5]; /*hash陣列*/
7 T h_base[MAXN+5]; /*h_base[n]=(prime^n)%mod*/
8 void hash_init(int len,T prime){
9     h_base[0]=1;
10     for(int i=1;i<len;++i){
11         h[i]=(h[i-1]*prime+s[i-1])%mod;
12         h_base[i]=(h_base[i-1]*prime)%mod;
13     }
14 }
15 T get_hash(int l,int r){ /*閉區間寫法 · 設編號
16     為 0 ~ len-1*/
17     return (h[r+1]-(h[l]*h_base[r-l+1])%mod+
18     mod)%mod;

```

10.7 minimal string rotation

```

1 int min_string_rotation(const string &s){
2     int n=s.size(),i=0,j=1,k=0;
3     while(i<n&&j<n&&k<n){
4         int t=s[(i+k)%n]-s[(j+k)%n];
5         ++k;
6         if(t){

```

```

7 |         if(t>0)i+=k;
8 |         else j+=k;
9 |         if(i==j)++;j;
10 |         k=0;
11 |     }
12 | }
13 | return min(i,j); //最小循環表示法起始位置
14 | }

```

10.8 reverseBWT

```

1 | const int MAXN = 305, MAXC = 'Z';
2 | int ranks[MAXN], tots[MAXC], first[MAXC];
3 | void rankBWT(const string &bw){
4 |     memset(ranks,0,sizeof(int)*bw.size());
5 |     memset(tots,0,sizeof(tots));
6 |     for(size_t i=0;i<bw.size();++i)
7 |         ranks[i] = tots[int(bw[i])]+1;
8 | }
9 | void firstCol(){
10 |     memset(first,0,sizeof(first));
11 |     int totc = 0;
12 |     for(int c='A';c<='Z';++c){
13 |         if(!tots[c]) continue;
14 |         first[c] = totc;
15 |         totc += tots[c];
16 |     }
17 | }
18 | string reverseBwt(string bw,int begin){
19 |     rankBWT(bw), firstCol();
20 |     int i = begin; //原字串最後一個元素的位置
21 |     string res;
22 |     do{
23 |         char c = bw[i];
24 |         res = c + res;
25 |         i = first[int(c)] + ranks[i];
26 |     }while( i != begin );
27 |     return res;
28 | }

```

11 default

11.1 debug

```

1 | #ifndef DEBUG
2 | #define dbg(...) {\
3 |     fprintf(stderr,"%s - %d : (%s) = ",
4 |         __PRETTY_FUNCTION__, __LINE__,#
5 |         __VA_ARGS__); \
6 |     _DO(__VA_ARGS__); \
7 | }
8 | template<typename I> void _DO(I&&x){cerr<<x
9 | <<endl;}
10 | template<typename I,typename...T> void _DO(I
11 | &&x,T&&...tail){cerr<<x<<" ";_DO(tail
12 | ...);}
13 | #else

```

```

9 | #define dbg(...)
10 | #endif

```

11.2 template

```

1 | // alias g++='g++ -std=c++14 -fsanitize=
2 | undefined -Wall -Wextra -Wshadow -D
3 | LOCAL'
4 |
5 | #include <bits/stdc++.h>
6 | using namespace std;
7 |
8 | #ifdef LOCAL
9 | #include "../debug.h"
10 | #else
11 | #pragma GCC optimize("O3,unroll-loops")
12 | #pragma GCC target("avx2,bmi,bmi2,lzcnt,
13 | popcnt")
14 | #define debug(...) ((void)0)
15 | #define orange(...) ((void)0)
16 | #endif
17 |
18 | #define int long long
19 | #define pii pair<int, int>
20 | #define ff first
21 | #define ss second
22 | #define pb push_back
23 | #define SPEEDY ios_base::sync_with_stdio(
24 | false); cin.tie(0); cout.tie(0);
25 |
26 | void solve() {
27 |
28 | }
29 |
30 | signed main() {
31 |     SPEEDY;
32 |
33 |     return 0;
34 | }

```

11.3 vimrc

```

1 | set nu
2 | set ts=4
3 | set shiftwidth=4
4 | set autoindent smartindent
5 | set cindent
6 |
7 | imap jk <ESC>
8 | inoremap {<CR> {<CR><ESC>ko

```

12 other

12.1 Nim game

```

1 | a1^a2^a3^...^an != 0 ? A win : B win

```

12.2 找小於 n 所有出現的 1 數量

```

1 | current == 0 higher * factor
2 | current == 1 higher * factor + lower + 1
3 | other current (higher + 1) * factor

```

13 other language

13.1 python heap

```

1 | import heapq
2 |
3 | heap = [7,1,2,2]
4 | heapq.heapify(heap)
5 | print(heap) # [1, 2, 2, 7]
6 | heapq.heappush(heap, 5)
7 | print(heap) # [1, 2, 2, 7, 5]
8 | print(heapq.heappop(heap)) # 1
9 | print(heap) # [2, 2, 5, 7]

```

13.2 java

13.2.1 文件操作

```

1 | import java.io.*;
2 | import java.util.*;
3 | import java.math.*;
4 | import java.text.*;
5 |
6 | public class Main{
7 |
8 |     public static void main(String args[]){
9 |         throws FileNotFoundException,
10 |         IOException
11 |         Scanner sc = new Scanner(new FileReader(
12 |             "a.in"));
13 |         PrintWriter pw = new PrintWriter(new
14 |             FileWriter("a.out"));
15 |         int n,m;
16 |         n=sc.nextInt();//讀入下一個INT
17 |         m=sc.nextInt();
18 |
19 |         for(ci=1; ci<=c; ++ci){
20 |             pw.println("Case #"+ci+": easy for
21 |                 output");
22 |         }
23 |
24 |         pw.close();//关闭流并释放，这个很重要，
25 |             否则是没有输出的
26 |         sc.close();//关闭流并释放
27 |     }
28 | }

```

13.2.2 优先队列

```

1 | PriorityQueue queue = new PriorityQueue( 1,
2 |     new Comparator(){
3 |         public int compare( Point a, Point b ){
4 |             if( a.x < b.x || a.x == b.x && a.y < b.y )
5 |                 return -1;
6 |             else if( a.x == b.x && a.y == b.y )
7 |                 return 0;
8 |             else return 1;
9 |         }
10 |     });

```

13.2.3 Map

```

1 | Map map = new HashMap();
2 | map.put("sa","dd");
3 | String str = map.get("sa").toString();
4 |
5 | for(Object obj : map.keySet()){
6 |     Object value = map.get(obj );
7 | }

```

13.2.4 sort

```

1 | static class cmp implements Comparator{
2 |     public int compare(Object o1,Object o2){
3 |         BigInteger b1=(BigInteger)o1;
4 |         BigInteger b2=(BigInteger)o2;
5 |         return b1.compareTo(b2);
6 |     }
7 | }
8 | public static void main(String[] args)
9 |     throws IOException{
10 |     Scanner cin = new Scanner(System.in);
11 |     int n;
12 |     n=cin.nextInt();
13 |     BigInteger[] seg = new BigInteger[n];
14 |     for (int i=0;i<n;i++)
15 |         seg[i]=cin.nextBigInteger();
16 |     Arrays.sort(seg,new cmp());

```

13.3 python output

```

1 hello = 'Hello'
2 world = 7122
3 print(f'{hello} {world}') # Hello 7122
4
5 import math
6 print(f'PI is approximately {math.pi:.3f}.')
7 # PI is approximately 3.142.
8
9 print('AAA {} BBB "{}!"'.format('Jin', 'Kela'))
10 # AAA Jin BBB "Kela!"
11
12 hello = 'hello, world\n'
13 hellos = repr(hello)
14 print(hellos) # 'hello, world\n'
15
16 x = 32.5
17 y = 40000
18 print(repr((x, y, ('spam', 'eggs'))))
19 # "(32.5, 40000, ('spam', 'eggs'))"
20
21 x = 7
22 print(eval('3 * x')) # 21

```

13.4 python 大數因數分解

```

1 # 大數因數分解 (使用 Pollard's Rho 與 Miller-Rabin)
2 import sys, random
3 from math import gcd
4
5 # Miller-Rabin 檢定 (機率性質數判定)
6 def is_probable_prime(n, k=12):
7     if n < 2:
8         return False
9     # 先檢查一些小質數
10    small_primes = [2,3,5,7,11,13,17,19,23,29]
11    for p in small_primes:
12        if n % p == 0:
13            return n == p
14    # 把 n-1 寫成 d * 2^s
15    d = n - 1
16    s = 0
17    while d % 2 == 0:
18        d //= 2
19        s += 1
20
21    # 重複 k 次隨機測試
22    for _ in range(k):
23        a = random.randrange(2, n - 1) # 隨機挑一個測試基數
24        x = pow(a, d, n)
25        if x == 1 or x == n - 1:
26            continue
27        composite = True
28        for __ in range(s - 1):
29            x = pow(x, 2, n)
30            if x == n - 1:
31                composite = False

```

```

31        break
32        if composite:
33            return False
34        return True
35
36 # Pollard's Rho 演算法 (找非平凡因數)
37 def pollards_rho(n):
38     if n % 2 == 0:
39         return 2
40     if n % 3 == 0:
41         return 3
42     # 隨機多項式 (x^2 + c) mod n
43     while True:
44         c = random.randrange(1, n-1) # 隨機挑選常數 c
45         x = random.randrange(2, n-1) # 起始點
46         y = x
47         d = 1
48         while d == 1:
49             x = (pow(x, 2, n) + c) % n # x -> f(x)
50             y = (pow(y, 2, n) + c) % n # y -> f(y) · 走兩步
51             y = (pow(y, 2, n) + c) % n
52             d = gcd(abs(x - y), n) # 計算兩者差的 gcd
53             if d == n:
54                 break
55             if d > 1 and d < n:
56                 return d
57
58 # 遞迴分解
59 def factor(n, out):
60     if n == 1:
61         return
62     if is_probable_prime(n):
63         out.append(n)
64     else:
65         d = pollards_rho(n)
66         while d is None or d == n: # 偶爾失敗就重試
67             d = pollards_rho(n)
68         factor(d, out)
69         factor(n // d, out)
70
71 def main():
72     data = sys.stdin.read().strip().split()
73     if not data:
74         return
75     # 每個 token 當作一個數字
76     for token in data:
77         try:
78             n = int(token)
79         except:
80             continue
81         if n <= 1:
82             print(n)
83             continue
84         facs = []
85         factor(n, facs)
86         facs.sort()

```

```

87         # 輸出因數
88         print(" ".join(str(x) for x in facs))
89
90 if __name__ == "__main__":
91     random.seed() # 使用系統時間作為隨機種子
92     main()

```

13.5 decimal

```

1 # 使用 decimal 模組來處理高精度小數運算
2 from decimal import *
3 setcontext(Context(prec=MAX_PREC, Emax=MAX_EMAX, rounding=ROUND_FLOOR))
4 print(Decimal(input()) * Decimal(input()))
5
6 # 將小數轉成分數 · 方便做近似或理論分析 · 且可以限制分母大小 ·
7 from fractions import Fraction
8 Fraction('3.14159').limit_denominator(10).numerator # 22
9
10 # 設定精確度
11 from decimal import Decimal, getcontext
12
13 # 精確位數設定
14 getcontext().prec = 70
15
16 n = 100
17 # 指定 n 為高精確度的物件
18 n = Decimal(n)
19 n /= 7
20 print(n)
21
22 # 將小數轉成分數
23 from fractions import Fraction
24
25 n = 1.5654
26 # 建立一個轉換物件
27 n = Fraction(n)
28
29 print(n)

```

13.6 python 大數排序

```

1 # 大數排序
2 # Line one n : 多少數字
3 # next line : 依序輸入每行一個
4 # sort : sort + Lambda
5 from sys import stdin
6
7 data = stdin.read().splitlines()
8
9 i = 0
10
11 while(i < len(data)):
12     T = int(data[i].strip())

```

```

13     i += 1
14     temp = [int(data[i + index].strip()) for index in range(T)]
15     i += T
16     temp = sorted(temp, key = lambda x : (x))
17     for ele in temp:
18         print(ele)

```

13.7 python 大數計算 2

```

1 # 單行輸入
2 # format : n1, operation, n2
3 from sys import stdin
4
5 data = stdin.read().splitlines()
6
7 limit = len(data)
8 i = 0
9
10 while(i < limit):
11     a, operation, b = map(str, data[i].split())
12     a, b = int(a), int(b)
13     i += 1
14     if(operation == '+'):
15         print(int(a + b))
16     elif(operation == '-'):
17         print(int(a - b))
18     elif(operation == '*'):
19         print(int(a * b))
20     else:
21         print(int(a // b))

```

13.8 python input

```

1 ans = sum(map(float, input().split()))
2 # input: 1.1 2.2 3.3 4.4 5.5
3 print(ans) # 16.5
4
5 (n, m) = map(int, input().split()) # 300 200
6 print(n * m) # 60000
7
8 Arr = list(map(int, input().split()))
9 # input: 1 2 3 4 5
10 print(Arr) # [1, 2, 3, 4, 5]

```

13.9 python 大數計算

```

1 # 讀取多行輸入
2 # Line one first number
3 # Line two operation
4 # Line three second number
5 from sys import stdin
6
7 data = stdin.read().splitlines()

```

```

8
9 limit = len(data)
10 i = 0
11 while(i < limit):
12     a = int(data[i].strip())
13     i += 1
14     operation = data[i].strip()
15     i += 1
16     b = int(data[i].strip())
17     i += 1
18     if(operation == '*'):
19         print(int(a * b))
20     else:
21         print(int(a / b))

```

14 zformula

14.1 formula

14.1.1 Pick 公式

給定頂點坐標均是整點的簡單多邊形，面積 = 內部格點數 + 邊上格點數/2-1

14.1.2 圖論

- 對於平面圖， $F = E - V + C + 1$ ， C 是連通分量數
- 對於平面圖， $E < 3V - 6$
- 對於連通圖 G ，最大獨立點集的大小設為 $I(G)$ ，最大匹配大小設為 $M(G)$ ，最小點覆蓋設為 $Cv(G)$ ，最小邊覆蓋設為 $Ce(G)$ 。對於任意連通圖：

- $I(G) + Cv(G) = |V|$
- $M(G) + Ce(G) = |V|$

- 對於連通二分圖：

- $I(G) = Cv(G)$
- $M(G) = Ce(G)$

- 最大權閉合圖：

- $C(u, v) = \infty, (u, v) \in E$
- $C(S, v) = W_v, W_v > 0$
- $C(v, T) = -W_v, W_v < 0$
- $ans = \sum_{W_v > 0} W_v - flow(S, T)$

- 最大密度子圖：

- 求 $max \left(\frac{W_e + W_u}{|V'|} \right), e \in E', v \in V'$
- $U = \sum_{v \in V} 2W_v + \sum_{e \in E} W_e$
- $C(u, v) = W_{(u, v)}, (u, v) \in E$ ，雙向邊
- $C(S, v) = U, v \in V$
- $D_u = \sum_{(u, v) \in E} W_{(u, v)}$
- $C(v, T) = U + 2g - D_v - 2W_v, v \in V$
- 二分搜 g ：
 $l = 0, r = U, eps = 1/n^2$
 if $((U \times |V| - flow(S, T))/2 > 0) l = mid$
 else $r = mid$
- $ans = min_cut(S, T)$
- $|E| = 0$ 要特殊判斷

- 弦圖：

- 點數大於 3 的環都要有一條弦
- 完美消除序列從後往前依次給每個點染色，給每個點染上可以染的最小顏色
- 最大團大小 = 色數
- 最大獨立集：完美消除序列從前往後能選就選
- 最小團覆蓋：最大獨立集的點和他延伸的邊構成
- 區間圖是弦圖
- 區間圖的完美消除序列：將區間按造又端點由小到大的排序
- 區間圖染色：用線段樹做

14.1.3 dinic 特殊圖複雜度

- 單位流： $O \left(min \left(V^{3/2}, E^{1/2} \right) E \right)$
- 二分圖： $O \left(V^{1/2} E \right)$

14.1.4 0-1 分數規劃

$x_i = \{0, 1\}$ ， x_i 可能會有其他限制，求 $max \left(\frac{\sum B_i x_i}{\sum C_i x_i} \right)$

- $D(i, g) = B_i - g \times C_i$
- $f(g) = \sum D(i, g) x_i$
- $f(g) = 0$ 時 g 為最佳解， $f(g) < 0$ 沒有意義
- 因為 $f(g)$ 單調可以二分搜 g
- 或用 Dinkelbach 通常比較快

```

1 binary_search(){
2     while(r-l>eps){
3         g=(l+r)/2;
4         for(i:所有元素)D[i]=B[i]-g*C[i];//D(i,g)
5         找出一組合法x[i]使f(g)最大;
6         if(f(g)>0) l=g;
7         else r=g;
8     }
9     Ans = r;
10 }
11 Dinkelbach(){
12     g=任意狀態(通常設為0);
13     do{
14         Ans=g;
15         for(i:所有元素)D[i]=B[i]-g*C[i];//D(i,g)
16         找出一組合法x[i]使f(g)最大;
17         p=0,q=0;
18         for(i:所有元素)
19             if(x[i])p+=B[i],q+=C[i];
20         g=p/q;//更新解，注意q=0的情況
21     }while(abs(Ans-g)>EPS);
22     return Ans;
23 }

```

14.1.5 學長公式

- $\sum_{d|n} \phi(n) = n$
- $g(n) = \sum_{d|n} f(d) \Rightarrow f(n) = \sum_{d|n} \mu(d) \times g(n/d)$
- Harmonic series $H_n = \ln(n) + \gamma + 1/(2n) - 1/(12n^2) + 1/(120n^4)$
- $\gamma = 0.57721566490153286060651209008240243104215$
- 格雷碼 $= n \oplus (n >> 1)$
- $SG(A + B) = SG(A) \oplus SG(B)$
- 選轉矩陣 $M(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$

14.1.6 基本數論

- $\sum_{d|n} \mu(n) = [n == 1]$
- $g(m) = \sum_{d|m} f(d) \Leftrightarrow f(m) = \sum_{d|m} \mu(d) \times g(m/d)$
- $\sum_{i=1}^n \sum_{j=1}^m \text{互質數量} = \sum \mu(d) \lfloor \frac{n}{d} \rfloor \lfloor \frac{m}{d} \rfloor$
- $\sum_{i=1}^n \sum_{j=1}^n lcm(i, j) = n \sum_{d|n} d \times \phi(d)$

14.1.7 排組公式

- k 卡特蘭 $\frac{C_n^{kn}}{n(k-1)+1} \cdot C_m^n = \frac{n!}{m!(n-m)!}$
- $H(n, m) \cong x_1 + x_2 \dots + x_n = k, num = C_k^{n+k-1}$
- Stirling number of $2^n d, n$ 人分 k 組方法數目

- $S(0, 0) = S(n, n) = 1$
- $S(n, 0) = 0$
- $S(n, k) = kS(n-1, k) + S(n-1, k-1)$

- Bell number, n 人分任意多組方法數目

- $B_0 = 1$
- $B_n = \sum_{i=0}^n S(n, i)$
- $B_{n+1} = \sum_{k=0}^n C_k^n B_k$
- $B_{p+n} \equiv B_n + B_{n+1} \pmod{p}$, p is prime
- $B_p^{m+n} \equiv mB_n + B_{n+1} \pmod{p}$, p is prime
- From $B_0 : 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, 115975$

- Derangement, 錯排，沒有人在自己位置上

- $D_n = n!(1 - \frac{1}{1!} + \frac{1}{2!} - \frac{1}{3!} \dots + (-1)^n \frac{1}{n!})$
- $D_n = (n-1)(D_{n-1} + D_{n-2}), D_0 = 1, D_1 = 0$
- From $D_0 : 1, 0, 1, 2, 9, 44, 265, 1854, 14833, 133496$

- Binomial Equality

- $\sum_k \binom{r}{m+k} \binom{s}{n-k} = \binom{r+s}{m+n}$
- $\sum_k \binom{m+k}{l} \binom{s}{n-k} = \binom{l+s}{l+n}$
- $\sum_k \binom{m+k}{l} \binom{s+k}{n-k} (-1)^k = (-1)^{l+m} \binom{s-m}{n-l}$
- $\sum_{k \leq l} \binom{l-k}{m} \binom{s-n}{k-n-m} (-1)^{l+m} \binom{s-m-1}{l-n-m} =$
- $\sum_{0 \leq k \leq l} \binom{l-k}{m} \binom{q+k}{n} = \binom{l+q+1}{m+n+1}$
- $\binom{r}{k} = (-1)^k \binom{k-r-1}{k}$

- $\binom{r}{m} \binom{m}{k} = \binom{r}{k} \binom{r-k}{m-k}$
- $\sum_{k \leq n} \binom{r+k}{k} = \binom{r+n+1}{n}$
- $\sum_{0 \leq k \leq n} \binom{k}{m} = \binom{n+1}{m+1}$
- $\sum_{k \leq m} \binom{m+r}{m+r-k} x^k y^k = \sum_{k \leq m} \binom{r}{k} (-x)^k (x+y)^{m-k}$

14.1.8 冪次, 冪次和

- $a^{b\%p} P = a^{b\% \varphi(p) + \varphi(p)}, b \geq \varphi(p)$
- $1^3 + 2^3 + 3^3 + \dots + n^3 = \frac{n^4}{4} + \frac{n^3}{2} + \frac{n^2}{4}$
- $1^4 + 2^4 + 3^4 + \dots + n^4 = \frac{n^5}{5} + \frac{n^4}{2} + \frac{n^3}{3} - \frac{n}{30}$
- $1^5 + 2^5 + 3^5 + \dots + n^5 = \frac{n^6}{6} + \frac{n^5}{2} + \frac{5n^4}{12} - \frac{n^2}{12}$
- $0^k + 1^k + 2^k + \dots + n^k = P(k), P(k) = \frac{(n+1)^{k+1} - \sum_{i=0}^{k-1} C_i^{k+1} P(i)}{k+1}, P(0) = n+1$
- $\sum_{k=0}^{m-1} k^n = \frac{1}{n+1} \sum_{k=0}^n C_k^{n+1} B_k m^{n+1-k}$
- $\sum_{j=0}^m C_j^{m+1} B_j = 0, B_0 = 1$
- 除了 $B_1 = -1/2$ ，剩下的奇數項都是 0
- $B_2 = 1/6, B_4 = -1/30, B_6 = 1/42, B_8 = -1/30, B_{10} = 5/66, B_{12} = -691/2730, B_{14} = 7/6, B_{16} = -3617/510, B_{18} = 43867/798, B_{20} = -174611/330,$

14.1.9 Burnside's lemma

- $|X/G| = \frac{1}{|G|} \sum_{g \in G} |X^g|$
- $X^g = t^{c(g)}$
- G 表示有幾種轉法， X^g 表示在那種轉法下，有幾種是會保持對稱的， t 是顏色數， $c(g)$ 是循環節不動的面數。
- 正立方體塗三顏色，轉 0 有 3^6 個元素不變，轉 90 有 6 種，每種有 3^3 不變，180 有 $3 \times 3^4 \cdot 120(\text{角})$ 有 $8 \times 3^2 \cdot 180(\text{邊})$ 有 $6 \times 3^3 \cdot \text{全部}$ $\frac{1}{24} (3^6 + 6 \times 3^3 + 3 \times 3^4 + 8 \times 3^2 + 6 \times 3^3) = 57$

14.1.10 Count on a tree

- Rooted tree: $s_{n+1} = \frac{1}{n} \sum_{i=1}^n (i \times a_i \times \sum_{j=1}^{\lfloor n/i \rfloor} a_{n+1-i \times j})$
- Unrooted tree:
 - Odd: $a_n - \sum_{i=1}^{n/2} a_i a_{n-i}$
 - Even: $Odd + \frac{1}{2} a_{n/2} (a_{n/2} + 1)$
- Spanning Tree
 - 完全圖 $n^n - 2$
 - 一般圖 (Kirchhoff's theorem) $M[i][i] = degree(V_i), M[i][j] = -1, \text{if have } E(i, j), 0 \text{ if no edge. delete any one row and col in } A, ans = det(A)$

14.1.11 循環小數轉分數

1. 若 $x = 0.\overline{a}$ ，則

$$x = \underbrace{\frac{a}{99 \dots 9}}_{k \text{ digits}}$$

其中 a 為循環節， k 為循環節的位數。

2. 例子：

$$0.\overline{37} = \frac{37}{99}$$

$$0.\overline{5} = \frac{5}{9}$$

14.1.12 循環小數轉分數

1. 純循環小數：若 $x = 0.\overline{a}$ ，其中 a 為循環節、長度為 k ，

$$x = \underbrace{\frac{a}{99 \dots 9}}_{k \text{ digits}}$$

例：

$$0.\overline{37} = \frac{37}{99}, \quad 0.\overline{5} = \frac{5}{9}$$

2. 混循環小數：若 $x = 0.b\overline{a}$ ，其中 b 為前綴、長度 m ， a 為循環節、長度 k ，

$$x = \frac{(b \cdot 10^k + a) - b}{10^m (10^k - 1)}$$

例：

$$0.12\overline{3} = \frac{(12 \cdot 10^1 + 3) - 12}{10^2 (10^1 - 1)} = \frac{123 - 12}{100 \cdot 9} = \frac{111}{900} = \frac{37}{300}$$

14.1.13 常見級數與組合公式

1. 平方和公式：

$$1^2 + 2^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6}$$

$$4^2 + 6^2 + \dots + (2n)^2 = \frac{(2n)(n+1)(2n+1)}{3}$$

$$1^2 + 3^2 + \dots + (2n+1)^2 = \frac{n(2n-1)(2n+1)}{3}$$

2. 立方和公式：

$$1^3 + 2^3 + \dots + n^3 = \frac{n^4 + 2n^3 + n^2}{4}$$

3. 四次方和公式：

$$1^4 + 2^4 + \dots + n^4 = \frac{6n^5 + 15n^4 + 10n^3 - n}{30}$$

4. 五次方和公式：

$$1^5 + 2^5 + \dots + n^5 = \frac{2n^6 + 6n^5 + 5n^4 - n^2}{12}$$

5. 六次方和公式：

$$1^6 + 2^6 + \dots + n^6 = \frac{6n^7 + 21n^6 + 21n^5 - 7n^3 + n}{42}$$

6. 七次方和公式：

$$1^7 + 2^7 + \dots + n^7 = \frac{3n^8 + 12n^7 + 14n^6 - 7n^4 + 2n^2}{24}$$

7. 八次方和公式：

$$\begin{aligned} & 1^8 + 2^8 + \dots + n^8 \\ &= \frac{10n^9 + 45n^8 + 60n^7 - 42n^5 + 20n^3 - 3n}{90} \end{aligned}$$

8. 九次方和公式：

$$\begin{aligned} & 1^9 + 2^9 + \dots + n^9 \\ &= \frac{2n^{10} + 10n^9 + 15n^8 - 14n^6 + 10n^4 - 3n^2}{20} \end{aligned}$$

9. 十次方和公式：

$$\begin{aligned} & 1^{10} + 2^{10} + \dots + n^{10} \\ &= \frac{6n^{11} + 33n^{10} + 55n^9 - 66n^7 + 66n^5 - 33n^3 + 5n}{66} \end{aligned}$$

10. 等比級數：

$$S = a \cdot \frac{r^n - 1}{r - 1}$$

11. 二項式係數恆等式：

$$\binom{n}{0} + \binom{n}{1} + \dots + \binom{n}{n} = 2^n$$

$$\binom{n}{1} + \binom{n}{2} + \dots + \binom{n}{n} = 2^n - 1$$

$$\binom{n}{1} + \binom{n}{3} + \dots + \binom{n}{n-k} = 2^{n-1}$$

12. 分配問題（玩具分給小孩）：

- (a) n 個玩具， k 位小孩，可以有人沒拿到：

$$\binom{n+k-1}{n} = \binom{n+k-1}{k-1}$$

- (b) n 個玩具， k 位小孩，每個人至少一個：

$$\binom{n-1}{k-1}$$

14.1.14 位元運算

- (a) 位元條件：

$$(x + k) \& (y + k) = 0$$

- (b) 加法恆等式（利用 XOR 與 AND）：

$$a + b = (a \oplus b) + 2 \cdot (a \& b)$$

- (c) OR 與 AND 的關係：

$$a \mid b = a + b - (a \& b)$$

- (d) 交換兩數：

$$a = a \oplus b, \quad b = a \oplus b, \quad a = a \oplus b$$

- (e) 取得最低位的 1：

$$x \& (-x)$$

- (f) 清除最低位的 1：

$$x \& (x - 1)$$

14.1.15 除數與倍數

- (a) LCM:

$$\text{lcm}(a, b) = \frac{a \times b}{\text{gcd}(a, b)}$$

- (b) LCM (prime factorization):

$$\text{lcm}(a, b) = \prod_{p \in \text{primes}} p^{\max(e_a, e_b)}$$

- (c) GCD (prime factorization):

$$\text{gcd}(a, b) = \prod_{p \in \text{primes}} p^{\min(e_a, e_b)}$$

- (d) Counting divisors:

$$\text{If } n = \prod_{i=1}^k p_i^{e_i}, \text{ cnt is } \prod_{i=1}^k (e_i + 1)$$

14.1.16 數論公式

13. Bezout's identity：

$$ax + by = \text{gcd}(a, b) \quad (\text{必定存在整數解 } x, y)$$

14. 模指數運算（冪的冪）：

$$a^{b^c} \equiv a^{\text{power_mod}(b, c, \text{MOD}-1)} \pmod{\text{MOD}}$$

Pythagorean theorem：

$$(n^2 + m^2, 2nm, n^2 - m^2), \quad n > m > 0$$

Wilson's theorem：

$$(p-1)! \pmod{p} = p-1 \quad (p \text{ 為質數})$$

Catalan number：

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \frac{(2n)!}{(n+1)!n!}$$

海龍公式：

$$\text{Area} = \sqrt{s(s-a)(s-b)(s-c)}, \quad s = \frac{a+b+c}{2}$$

兩圓相交弦長：

$$\text{Length} = 2 \cdot \sqrt{\frac{(s-a)(s-b)}{s}}, \quad s = \frac{r_1 + r_2 + d}{2}$$

三角形內切圓半徑：

$$r = \frac{2 \cdot \text{Area}}{a+b+c}$$

三角形外接圓半徑：

$$R = \frac{abc}{4 \cdot \text{Area}}$$

三角形重心：

$$G = \left(\frac{x_1 + x_2 + x_3}{3}, \frac{y_1 + y_2 + y_3}{3} \right)$$

三角形垂心：

$$H = \left(\frac{x_1 \tan A + x_2 \tan B + x_3 \tan C}{\tan A + \tan B + \tan C}, \frac{y_1 \tan A + y_2 \tan B + y_3 \tan C}{\tan A + \tan B + \tan C} \right)$$

三角形內心：

$$I = \left(\frac{ax_1 + bx_2 + cx_3}{a+b+c}, \frac{ay_1 + by_2 + cy_3}{a+b+c} \right)$$

奇(偶)長度陣列子數量：

$$n = \frac{(i+1) * (n-i) + 1}{2}$$

15 Интернационал

15.1 保佑

```
1
2
3
4 //      _ooθoo_
5 //      o8888888o
6 //      88" . "88
7 //      (| -_- |)
8 //      0\  =  /θ
9 //
10 //      /--\
11 //      |   |
12 //      |   |
13 //      |   |
14 //      |   |
15 //      |   |
16 //      |   |
17 //      |   |
18 //      |   |
19 //      |   |
20 //      |   |
21 //      |   |
22 //      |   |
23 //      |   |
24 //      |   |
25 //      |   |
26 //      |   |
27 //      |   |
28 //      |   |
29 //      |   |
30 //      |   |
31 //      |   |
32 //      |   |
33 //      |   |
34 //      |   |
35 //      |   |
36 //      |   |
37 //      |   |
38 //      |   |
39 //      |   |
40 //      |   |
41 //      |   |
42 //      |   |
43 //      |   |
44 //      |   |
45 //      |   |
46 //      |   |
47 //      |   |
48 //      |   |
49 //      |   |
50 //      |   |
51 //      |   |
52 //      |   |
53 //      |   |
54 //      |   |
55 //      |   |
```

```
56
57
58 // ##      #####
59 // ##      ##
60 // ##      ##
61 // ##      ##
62 // ##      ##
63 // ##      ##
64 // ##      ##
65 // ##      ##
66 // #####
67 //      ##      ##
68 //      ##      ##
69 //      ##      ##
70 //      ##      ##
71 //      ##      ##
72 //      ##      ##
73 //      ##      ##
74 // #####      ##
75 //
76 //      元首保佑 永無BUG
77
78 //
79 //      < @ \
80 //      \_/_/_/ \_/_/_/ \_/_/_/
81 //      \_/_/_/ \_/_/_/ \_/_/_/
82 //      \_/_/_/ \_/_/_/ \_/_/_/
83 //      u/u/u/ \_/_/_/ \_/_/_/
84 //      u/u/ \_/_/_/ \_/_/_/
85 //      | | |
86 //      | |
87 //      /+--+ \
88 //      || 5 ||
89 //      \|=//
90 //
91 //      神獸保佑 永無BUG
```

```
34 #
35 #
36 #
37 #
38 #
39 #
40 #
41 #
42 #
43 #
44 #
45 #
46 #
47 #
48 #
49 #
50 #
51 #
52 #
53 #
54 #
55 #
```

ACM ICPC Team Reference - BogoSort

Contents

1 Algorithm	1	4 Data Structure	5	6.18 all longest path	14	10.6 hash	19
1.1 LIS	1	4.1 undo disjoint set	5	6.19 stable matching	14	10.7 minimal string rotation	19
1.2 LCS	1	4.2 segment tree range update (lazy propagation)	5	6.20 tree diameter (len,end)	14	10.8 reverseBWT	20
2 Basic	1	4.3 segment tree prefix sum lower bound	6	7 Language	14	11 default	20
2.1 mod helper function	1	4.4 Fenwick tree (BIT)	6	7.1 CNF	14	11.1 debug	20
2.2 self-defined-pq-operator	1	4.5 Trie (Prefix tree)	6	8 Number Theory	15	11.2 template	20
2.3 generating all subsets	1	4.6 segment tree	6	8.1 Linear Sieve	15	11.3 vimrc	20
2.4 memset	1	4.7 BIT range update point query . .	6	8.2 C(n,m)	15	12 other	20
2.5 submask enumeration	1	4.8 DSU remove node find prev next one	7	8.3 Euler Totient 1 to n	15	12.1 Nim game	20
2.6 custom-hash	1	4.9 DSU	7	8.4 derangement (Principle of Inclusion-Exclusion)	15	12.2 找小於 n 所有出現的 1 數量 . .	20
2.7 stringstream split by comma . .	1	5 Geometry	7	8.5 matrix template (with fast power)	15	13 other language	20
3 DP	1	5.1 cross. product	7	8.6 Sieve of Eratosthenes (with big num)	15	13.1 python heap	20
3.1 deque	1	5.2 Check Point in Polygon	7	8.7 mod inv	15	13.2 java	20
3.2 walk	1	5.3 Point	7	8.8 derangement (DP)	15	13.2.1 文件操作	20
3.3 grouping	2	5.4 Polygon Lattice Points	8	8.9 Euler Totient	16	13.2.2 優先隊列	20
3.4 matching	2	5.5 line intersect	8	8.10 fast power	16	13.2.3 Map	20
3.5 projects	2	5.6 Polygon area	8	8.11 first and second mex	16	13.2.4 sort	20
3.6 stones	2	5.7 using std::complex	8	8.12 Catalan Number	16	13.3 python output	21
3.7 coins	2	5.8 point distance from a line . . .	9	8.13 Chinese Remainder Theorem . .	16	13.4 python 大數因數分解	21
3.8 elevator rides	3	6 Graph	9	8.14 mod inv (not prime)	16	13.5 decimal	21
3.9 slimes	3	6.1 Euler tour+RMQ	9	8.15 mod inv (not coprime)	16	13.6 python 大數排序	21
3.10 digit sum	3	6.2 Prim	9	8.16 Mint	16	13.7 python 大數計算 2	21
3.11 sushi	3	6.3 Eulerian cycle	9	8.17 C(n,k) mod inverse	16	13.8 python input	21
3.12 candies	3	6.4 maximum weight bipartite matching	10	8.18 Linear Diophantine 丟番圖 . .	16	13.9 python 大數計算	21
3.13 permutation	4	6.5 maximum flow	10	8.19 擴展歐基里德	16	14 zformula	22
3.14 Knasack2	4	6.6 maximum cardinality bipartite matching	11	8.20 Sieve of Eratosthenes	16	14.1 formula	22
3.15 flowers	4	6.7 Floyd-Warshall	11	8.21 builtin	17	14.1.1 Pick 公式	22
3.16 independent set	4	6.8 MST	11	8.22 C(n,k) DP	17	14.1.2 圖論	22
3.17 couting tower	4	6.9 Hopcroft-Karp	11	9 Range Queries	17	14.1.3 dinic 特殊圖複雜度	22
3.18 couting numbers	5	6.10 all longest path dfs	11	9.1 subarray maximum sum queries .	17	14.1.4 0-1 分數規劃	22
		6.11 all longest path top sort . . .	12	9.2 find first equal or greater . . .	18	14.1.5 學長公式	22
		6.12 Dijkstra	12	9.3 find k-th and count less than . .	18	14.1.6 基本數論	22
		6.13 Heavy-light decomposition (HLD)	12	10 String	18	14.1.7 排組公式	22
		6.14 binary lifting	12	10.1 Z	18	14.1.8 冪次, 冪次和	22
		6.15 maximum cardinality matching .	13	10.2 manacher	18	14.1.9 Burnside's lemma	22
		6.16 topological sort	13	10.3 AC 自動機	19	14.1.10 Count on a tree	22
		6.17 tree diameter	13	10.4 KMP	19	14.1.11 循環小數轉分數	23
				10.5 suffix array lcp	19	14.1.12 循環小數轉分數	23
						14.1.13 常見級數與組合公式 . . .	23
						14.1.14 位元運算	23
						14.1.15 除數與倍數	23
						14.1.16 數論公式	23

15	Интернационал	24	15.1 保佑	24
-----------	----------------------	-----------	-------------------	----

ACM ICPC Judge Test - BogoSort

C++ Resource Test

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 namespace system_test {
5
6     const size_t KB = 1024;
7     const size_t MB = KB * 1024;
8     const size_t GB = MB * 1024;
9
10    size_t block_size, bound;
11    void stack_size_dfs(size_t depth = 1) {

```

```

12        if (depth >= bound)
13            return;
14        int8_t ptr[block_size]; // 若無法編譯將
15            block_size 改成常數
16        memset(ptr, 'a', block_size);
17        cout << depth << endl;
18        stack_size_dfs(depth + 1);
19    }
20    void stack_size_and_runtime_error(size_t
21        block_size, size_t bound = 1024) {
22        system_test::block_size = block_size;
23        system_test::bound = bound;
24        stack_size_dfs();
25    }
26    double speed(int iter_num) {
27        const int block_size = 1024;
28        volatile int A[block_size];
29        auto begin = chrono::high_resolution_clock
30            ::now();
31        while (iter_num--)
32            for (int j = 0; j < block_size; ++j)
33                A[j] += j;
34        auto end = chrono::high_resolution_clock::
35            now();
36        chrono::duration<double> diff = end -
37            begin;

```

```

38        return diff.count();
39    }
40    void runtime_error_1() {
41        // Segmentation fault
42        int *ptr = nullptr;
43        *(ptr + 7122) = 7122;
44    }
45    void runtime_error_2() {
46        // Segmentation fault
47        int *ptr = (int *)memset;
48        *ptr = 7122;
49    }
50    void runtime_error_3() {
51        // munmap_chunk(): invalid pointer
52        int *ptr = (int *)memset;
53        delete ptr;
54    }
55    void runtime_error_4() {
56        // free(): invalid pointer
57        int *ptr = new int[7122];
58        ptr += 1;
59        delete[] ptr;
60    }
61    }
62

```

```

63    void runtime_error_5() {
64        // maybe illegal instruction
65        int a = 7122, b = 0;
66        cout << (a / b) << endl;
67    }
68    void runtime_error_6() {
69        // floating point exception
70        volatile int a = 7122, b = 0;
71        cout << (a / b) << endl;
72    }
73    void runtime_error_7() {
74        // call to abort.
75        assert(false);
76    }
77    } // namespace system_test
78
79    #include <sys/resource.h>
80    void print_stack_limit() { // only work in
81        Linux
82        struct rlimit l;
83        getrlimit(RLIMIT_STACK, &l);
84        cout << "stack_size = " << l.rlim_cur << "
85            byte" << endl;
86    }
87

```